

SATA Synthetic Experiment Pipeline

Design Specification v3 — data, presentation, selection, inference, SATA and probes
PyTorch, 7–8B models

SATA Project — Honours Thesis

26 September 2026 (revised 28 September 2026)

Contents

I	Overview	4
1	Purpose, scope and how to read this document	4
2	Research questions and hypotheses	4
3	Pipeline overview	5
4	Implementation status	6
5	Notation	6
6	Sources and attribution	7
II	Data generation	9
7	Configuration	9
8	Feature roles and column-role permutation	9
9	Task sampling	10
10	Rule families	10
10.1	Linear	11
10.2	Tree (signed majority)	11
10.3	Families excluded from evaluation	12
11	Environment generation	12
11.1	id — in distribution	12
11.2	covariate — $P(X)$ shift	12
11.3	Covariate probes (exploratory, added after P4)	13
11.4	spurious_reversal	14
11.5	mechanism — why it is a placebo for in-context learning	14
11.6	extrapolation and missing_feature	15
12	Spurious feature construction (f_8)	15
13	Label noise and row metadata	16
14	Dataset construction	16

15 Causal graph	17
III Presentation: naming, serialisation and prompts	17
16 Naming protocol	17
17 Serialisation	19
18 Prompt templates	19
IV Demonstration selection	21
19 Selection engine	21
20 Mechanisms	21
20.1 Counter-spurious selection must match on $ f_8 $	22
21 Seeding, reuse and pairing	23
22 Pool provenance and leakage	24
23 Demonstrations without the shortcut (exploratory, added 5 October 2026)	24
V Model inference	26
24 Models and runtime	26
25 Scoring	27
26 Prefix caching	27
27 Contextual calibration	28
28 Measuring priors	28
VI Evaluation and statistics	31
29 Metrics	31
30 Design and inference	31
31 Faithfulness and reliance (RQ3)	32
VII SATA	34
32 Architecture	34
33 Training targets	34
34 Training	35
35 Validation	35

36 Leakage and fairness controls	36
VIII Mechanistic probes	37
37 Residual stream and logit lens	37
38 Critical layer, probes and steering	37
IX Experiment orchestration	38
39 Grid, runs and compute	38
40 Gates	40
X Known issues in the v2 implementation	41
41 Bug-hunting notes (v3 invariants to test)	42
References	44
A Lexicons	53
B The eight admissible tree tables	54

Part I

Overview

1 Purpose, scope and how to read this document

This document specifies the complete synthetic experiment. It covers how tasks and data are generated, how rows are presented to the language model (feature naming, serialisation, prompts), how demonstrations are selected, how the model is queried, how results are evaluated, how SATA is trained, and how the model’s internal computation is probed. Every design choice is stated with its rationale. Where a property can be proved (label directions, base rates, invariances, freedom from leakage), the proof is given; where a property is only checked empirically, the check and its threshold are given.

The companion document `docs/research_plan.md` gives the phase schedule (P0–P7), gates and reading list. Section numbers there and here are independent; phases are referred to as P0–P7 in both. Section 6 lists the published work that each part of the design draws on; citations in the text give the details.

Version history. v2 (22 September 2026) documented the pipeline as then implemented, including several defects that invalidate its results; they are listed in Part X. v3 (this version) specifies the redesigned experiment for 7–8B models, run in PyTorch on Apple’s Metal (MPS) backend. The 27 September revision adds the sources of each design element (Section 6) and a bibliography, and drops the MLX fallback: PyTorch is the only backend. The 28 September revision records the P1 implementation: status labels, the refinements the implementation made to the mechanisms and seeding, the rendered prompt, the results schema and the G1 results.

Status labels. Each section carries one of three labels:

- **Implemented** — the current code on branch `synthetic-experiments` matches this section.
- **Planned (P n)** — specified here and implemented in phase n of the research plan.
- **Changes current code (P n)** — the current code differs; this section is the target, and the defect is listed in Part X.

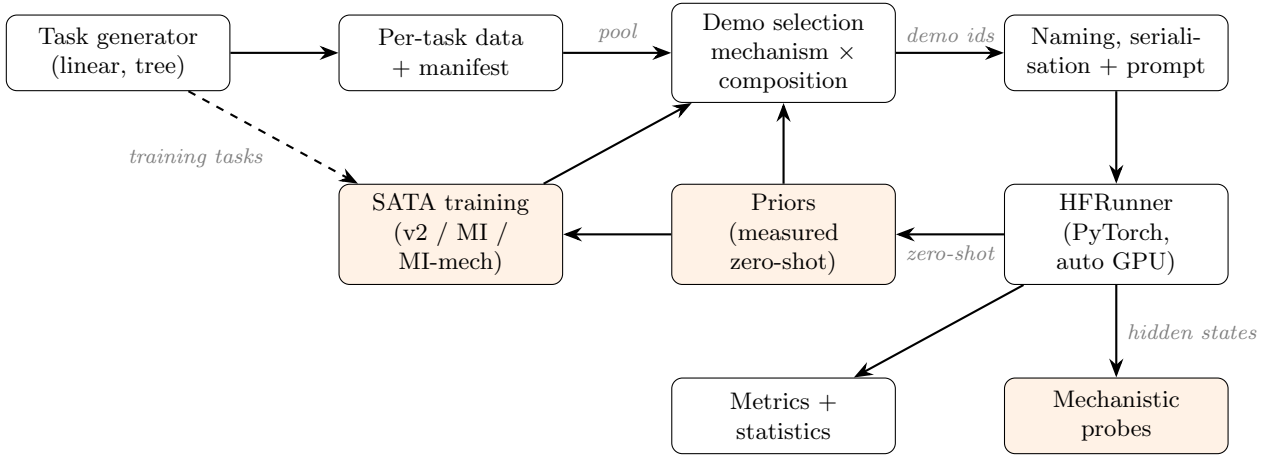
2 Research questions and hypotheses

Status: Planned (P4–P6)

In-context learning (Brown et al., 2020; Dong et al., 2024) adapts a language model to a task from demonstrations in its prompt. Wei et al. (2023) found that the ability to follow in-context evidence over semantic priors emerges with model scale. This thesis asks whether demonstration selection, and in particular a learned selector (SATA) trained with signals taken from the model itself, can help 7–8B models do so under controlled distribution shift. Evidence so far suggests 7–8B models follow contradicting demonstrations only partially: Llama-3.1-8B-Instruct agrees with fully flipped sentiment labels about half the time (Krishna Kumar, 2025), and with flipped labels in-context accuracy plateaus below its level with the original labels (Kossen et al., 2024). Models of this size also contain machinery for following the context: under false demonstrations their later layers switch to the demonstrated labels (Halawi et al., 2024), and a single direction in the residual stream controls whether they follow the context or their prior (Minder et al., 2025). The framing is therefore a graded contest between prior and demonstrations, in the spirit of Bayesian accounts of in-context learning (Xie et al., 2022), rather than an absent ability.

RQ	Question	Hypothesis
RQ1	Does in-context learning degrade out of distribution?	H1: calibrated balanced accuracy with random demonstrations is lower on covariate-shift, spurious-reversal and concept-shift (flipped-name) queries than on ID queries.
RQ2	Do different selection strategies win under different shift types?	H2: strategy $\times$ shift interaction; counter-spurious helps most under spurious reversal, coverage/kNN under covariate shift, counter-prior under concept shift.
RQ3	Does the model rely on the correct features, and why does the prior win when it does?	H3: under flipped names, behavioural reliance follows the demonstrations less than under abstract names while self-reports follow the names; the logit lens shows a layer where prior and demonstration evidence diverge.
RQ4	Does SATA beat hand-designed strategies, and do model-informed signals help?	H4: SATA-v2 is at least as good as the best hand-designed strategy on worst-environment accuracy; SATA-MI beats SATA-v2 and the SATA-names control under flipped names.

3 Pipeline overview



4 Implementation status

Component	v2 code	v3	Phase
Task generation	5 tasks; families linear, threshold, tree; 3–5 causal	24 evaluation tasks (+12 pilot); linear + tree; exactly 3 causal; manifest	done (P1)
Datasets	5 tasks merged into one 1,280-row pool per shift	per-task pools; standardised; column roles permuted	done (P1)
Naming	abstract only	abstract / aligned / flipped; screened lexicons	done (P3)
Selection	7 selectors, one fixed seed, test leakage	mechanism $\times$ composition engine; ground-truth tags; seeded demo sets	done (P1)
Prompt	ungrammatical template	one template (instruct and base)	done (P1)
Inference	MLXRunner; double BOS on Llama	HFRunner (PyTorch only); prefix caching	done (P1)
Device selection	CUDA-or-CPU in two places; SATA selection assumed CPU	shared auto-detection: CUDA, then MPS, then CPU	done (P0)
Calibration	none	contextual calibration	done (P1)
Demo masks	none	shortcut (and noise) feature hidden from demonstrations and selection; queries complete	built (5 Oct); run pending
Priors	none	measured zero-shot priors on the pool; surrogate (descriptive); conflict statistic	done (P3)
SATA	not on this branch	ported (v2) + MI + MI-mech	P5
Probes	none	logit lens, probes, steering	P6
Statistics	per-cell accuracy, $n = 30$	hierarchical bootstrap; Holm over a frozen contrast family	P2, P4

5 Notation

Symbol	Meaning
$x \in \mathbb{R}^{10}$	feature vector (generator scale unless stated)
$C \subset \{0, \dots, 7\}, C = 3$	load-bearing (causal) features of a task
$s \in [0.80, 0.90]$	spurious strength; $d = \Phi^{-1}(s)$
$\eta = 0.02$	label-noise rate
y_{clean}, y	rule label before and after label noise
$s_j \in \{+1, -1\}$	direction of load-bearing feature j
k	demonstrations per prompt (chosen in P2 from $\{8, 16, 32\}$)
$m = \ell_1 - \ell_0$	label margin (logit of 1 minus logit of 0)
$m_0(x; N)$	zero-shot margin under naming N
β_{name}	prior slope of a feature name (P3 surrogate, descriptive)
$\tilde{m}_0(x_i; N)$	measured zero-shot margin of pool row i , centred on the pool median
π_t	column-role permutation of task t
T, n	number of tasks, queries per task and environment

6 Sources and attribution

This design builds on published work. The table maps each part of the design to the work it draws on and what is taken from it; citations in later sections give the details. The starting points were the project’s literature review and the reading list in `docs/research_plan.md`; further sources were found by targeted searches, and every reference was checked against the original publication.

Component	What is taken, and from where
Research question	Following in-context evidence over semantic priors emerges with scale (Wei et al., 2023). 7–8B models follow contradicting demonstrations only partially (Kossen et al., 2024; Krishna Kumar, 2025), yet already contain machinery for following the context (Halawi et al., 2024; Minder et al., 2025). The contest between prior and evidence follows Bayesian accounts of in-context learning (Xie et al., 2022).
Synthetic tasks	Tasks sampled from known function classes, including linear functions and sign-split decision trees over Gaussian inputs (Garg et al., 2022); synthetic task priors for tabular models (Müller et al., 2022; Hollmann et al., 2023, 2025). Public tabular datasets can be memorised by LLMs (Bordt et al., 2024), which synthetic tasks avoid.
Shift types	Definitions of covariate, prior-probability and concept shift (Shimodaira, 2000; Quiñero-Candela et al., 2009; Moreno-Torres et al., 2012); most large performance drops on real tabular data are dominated by concept ($Y \mid X$) shift (Liu et al., 2023); in-context learners of linear functions are brittle under covariate shift (Zhang et al., 2024). Realising concept shift through naming is this project’s design.
Spurious feature	Colored MNIST: a noisy causal signal competing with a spurious feature whose correlation reverses at test time (Arjovsky et al., 2019); class-conditional Gaussian environmental features (Rosenfeld et al., 2021); why learners adopt spurious features and shortcuts (Nagarajan et al., 2021; Geirhos et al., 2020).
Naming	Flipped and semantically unrelated labels (Wei et al., 2023); named versus anonymised features in in-context regression (Nafar et al., 2025); task recognition versus task learning (Pan et al., 2023); the effect of feature names (Dinh et al., 2022; Choi et al., 2022). Applying the flip to feature names, with antonym pairs matched on measured prior strength, is this project’s design.
Serialisation, scoring	Rows serialised as text (Hegselmann et al., 2023; Dinh et al., 2022); classification by label-token probabilities (Brown et al., 2020); surface-form competition (Holtzman et al., 2021).
Selection strategies	The label, feature-range, rule and counter-spurious protocols come from the project’s literature review. Related work: similarity retrieval (Liu et al., 2022); diverse and coverage-based selection (Levy et al., 2023; Gupta et al., 2023); label-flipped comparable demonstrations (Fan et al., 2024); misconfidence-based selection (Xu and Zhang, 2024). Label balance and order are controlled because they bias predictions (Zhao et al., 2021; Min et al., 2022; Lu et al., 2022).
Counter-spurious sets	(label, spurious attribute) groups and worst-group accuracy (Sagawa et al., 2020), for which simple group balancing is competitive (Idrissi et al., 2022); nearest-neighbour matching (Rubin, 1973; Stuart, 2010); spurious features and shortcuts in in-context learning (Si et al., 2023; Tang et al., 2023; Harutyunyan et al., 2026; Chen et al., 2026). The magnitude-matching rule and its analysis are this project’s.
Demonstrations without the shortcut	A reference model trained without the spurious feature, as Colored MNIST’s grayscale oracle (Arjovsky et al., 2019); ablating parts of the demonstrations while the test input is kept (Min et al., 2022); removing the competing feature removes the underspecification studied by Si et al. (2023). On tabular shifts, predictors restricted to causal features did not generalise better than predictors using all features (Nastl and Hardt, 2024). Hiding features from demonstrations but not from queries is this project’s (Section 23).
Inference	Contextual calibration (Zhao et al., 2021); key–value and prefix caching (Pope et al., 2023; Gim et al., 2024; Zheng et al., 2024); causal attention (Vaswani et al., 2017).

Component	What is taken, and from where
Priors	Surrogate models (Craven and Shavlik, 1996; Ribeiro et al., 2016) fitted by ridge regression (Hoerl and Kennard, 1970); prior-data conflict (Evans and Moshonov, 2006) and knowledge conflicts in LLMs (Longpre et al., 2021; Xie et al., 2024); the random-label control (Min et al., 2022).
Faithfulness (RQ3)	Self-reported versus ablation-based feature rankings (Li et al., 2025); keeping perturbed inputs in distribution (Zhu et al., 2026), here by hot-deck imputation (Andridge and Little, 2010); removal-based importance (Breiman, 2001; Covert et al., 2021); LLM self-explanations and their unfaithfulness (Huang et al., 2023; Turpin et al., 2023); ground-truth importance from synthetic data (Liu et al., 2021). The demo-following index is this project’s.
Statistics	AUROC (Hanley and McNeil, 1982; Fawcett, 2006); balanced accuracy (Brodersen et al., 2010); shift gap (Gardner et al., 2023); bootstrap (Efron and Tibshirani, 1993; Davison and Hinkley, 1997; Saravanan et al., 2020); sign test (Dixon and Mood, 1946); Holm correction (Holm, 1979); pre-registration (Nosek et al., 2018); two-stage sampling variance (Cochran, 1977).
SATA	Attention over context examples to adapt to an environment (Gupta et al., 2024), as proposed in the literature review; row tokens carrying features and a label embedding (Hollmann et al., 2023) and attention over sets (Lee et al., 2019); selectors trained from language-model feedback (Rubin et al., 2022; Li et al., 2023; Ye et al., 2023; Zhang et al., 2022); temperature-softened targets (Hinton et al., 2015); task diversity (Goddard et al., 2025).
Model-informed targets	Datamodels (Ilyas et al., 2022), which Chang and Jia (2023) fit to in-context examples on the correct-class margin, the utility used here; subset-based influence estimates (Nguyen and Wong, 2023); selection signals from model internals or learned judges (Kato et al., 2025; Wang et al., 2026).
Mechanistic probes	Logit lens and its limits (nostalgebraist, 2020; Belrose et al., 2023); residual stream (Elhage et al., 2021); critical layer (Halawi et al., 2024); linear probes (Alain and Bengio, 2017; Belinkov, 2022); steering vectors (Turner et al., 2023; Rimsky et al., 2024); a context-versus-prior direction (Minder et al., 2025); task and function vectors (Hendel et al., 2023; Todd et al., 2024).
Supporting mathematics	Boolean functions (O’Donnell, 2014); Bayes error (Devroye et al., 1996); label noise (Angluin and Laird, 1988); causal graphs, anticausal features and mechanisms (Pearl, 2009; Schölkopf et al., 2012; Peters et al., 2017).
Models and software	Qwen et al. (2024); Grattafiori et al. (2024); Paszke et al. (2019); Wolf et al. (2020); Apple Inc. (2026); Harris et al. (2020); Pedregosa et al. (2011); Chen and Guestrin (2016); Reimers and Gurevych (2019).

Own design. Everything without a citation is this project’s own work, although it builds on the ideas above. This includes the naming lexicons and screening, the magnitude-matched counter-spurious rule, the analysis of the mechanism environment as a placebo, the column-role permutation, the SATA targets and model-informed variants, and the gates. The proofs are our own derivations; where they rest on a standard result, the source is cited at that point.

Part II

Data generation

7 Configuration

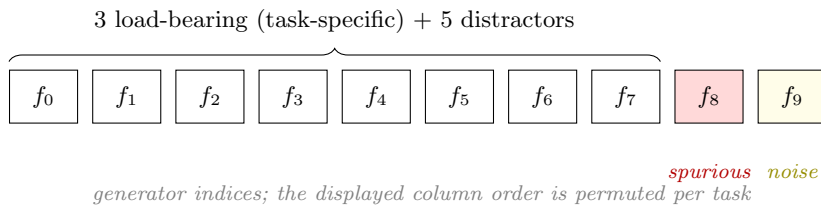
Status: Implemented (P1)

Parameter	v3 value	v2 value
<code>n_features</code>	10	10
load-bearing features $ C $	3	$\sim U\{3, 4, 5\}$
rule families	linear, tree	linear, threshold, tree
linear coefficients	$c_j = u_j r_j$, $u_j \sim U(1, 2)$, r_j Rademacher	$c_j \sim \mathcal{N}(0, 1.5^2)$
spurious strength s	$\sim U(0.80, 0.90)$	same
spurious direction σ_t	+1 or -1, half of each (family, domain) cell	always +1
label noise η	0.02	same
pool per task	256 rows	256
ID test per task	100 rows	100
OOD test per task and environment	100 rows	100
queries used per task and environment	$n = 20$ (10 per class)	first 30 of the merged set
evaluation tasks (suite <code>eval</code>)	24 (12 linear, 12 tree; 12 per domain)	5, merged
pilot tasks (suite <code>pilot</code>)	12 (6 linear, 6 tree), disjoint ids; P2 only	—
task seed	42: task i of suite S draws from <code>SeedSequence([42, S, i])</code>	global RNG state
SATA tasks	train and validation, disjoint ids (default 1000 / 100)	—
evaluated environments	id, covariate, spurious_reversal	covariate, mechanism, spurious_reversal

All values live in `configs/default.yaml`. The generator also implements three environments that are not evaluated: `mechanism`, `extrapolation` and `missing_feature`. Section 11.5 explains why `mechanism` is uninformative for in-context learning; it is used in SATA training. `scripts/make_synthetic_data.py` (or NB01) writes both suites; tasks and data are fully determined by the configuration.

8 Feature roles and column-role permutation

Status: Implemented (P1)



Inside the generator, load-bearing features are drawn from indices $\{0, \dots, 7\}$, the spurious feature is index 8 and the noise feature is index 9. When a task is written to disk, its 10 columns are reordered

by a permutation π_t drawn uniformly per task and recorded in the manifest. Every downstream component reads column roles from the manifest, never from position.

Lemma 8.1 (No positional shortcut). *If π_t is uniform on the permutations of $\{0, \dots, 9\}$ and independent of the task’s rule, the displayed position $\pi_t(8)$ of the spurious feature is uniform on $\{0, \dots, 9\}$ and independent of the rule. A selector trained across tasks (SATA) therefore cannot identify the spurious column by its position.*

Proof. For any position p , the number of permutations with $\pi(8) = p$ is $9!$ for every p , so $P(\pi_t(8) = p) = 9!/10! = 1/10$. Independence holds because π_t is drawn independently of the rule. $\square$

In v2 the spurious feature was always f_8 (the ninth column), which a selector trained across tasks can learn by position: a shortcut in the sense of Geirhos et al. (2020).

9 Task sampling

Status: Implemented (P1)

Sampling tasks from known function classes follows studies of in-context learning on synthetic functions (Garg et al., 2022) and the synthetic priors used to train tabular foundation models (Müller et al., 2022; Hollmann et al., 2023, 2025). Synthetic tasks also avoid the memorisation of public tabular datasets documented for large language models such as GPT-3.5 and GPT-4 (Bordt et al., 2024), and they provide the ground truth (load-bearing features, directions and spurious strength) that the analyses need.

Each task i of a suite S is sampled by `sample_eval_tasks` from its own stream (a NumPy `SeedSequence` of the task seed, the suite name and i), so its parameters do not depend on the other tasks, except its family and its spurious direction, which are balanced over the suite:

1. **Rule family:** fixed by position (the first half of the suite linear, the rest tree: tasks 0–11 and 12–23 in `eval`), crossed with domain, which alternates (loan, medical), so each family has 6 tasks per domain (Section 16).
2. **Load-bearing features:** 3 distinct indices from $\{0, \dots, 7\}$, sorted.
3. **Linear coefficients:** $c_j = u_j r_j$ with $u_j \sim U(1, 2)$ and r_j a Rademacher sign, so every load-bearing feature has a non-negligible effect and a definite direction.
4. **Tree leaf table:** the signed majority with literal signs r_j , which is uniform over the 8 admissible tables (Theorem 10.2); the code asserts that each table is complement antisymmetric and not degenerate.
5. **Spurious strength:** $s \sim U(0.80, 0.90)$.
6. **Spurious direction:** $\sigma_t \in \{+1, -1\}$ (Section 12), $+1$ for half of the tasks in each (family, domain) cell and -1 for the other half, in random order, from a separate stream so the other draws are unchanged. An odd cell’s extra sign alternates between cells, so each pilot family (two cells of 3 tasks) is still split 3/3.
7. **Data seed:** one integer from the same stream, from which every split of the task derives (Section 14).

Two suites are written: `eval` (24 tasks) and `pilot` (12 tasks), from the same distribution with disjoint task ids. The pilot suite is used only in P2, so the number of demonstrations is chosen without touching the evaluation tasks. SATA training and validation tasks will be drawn from the same distribution with disjoint ids (P5).

10 Rule families

Status: Implemented (P1): evaluation uses linear and tree only

Given $X \in \mathbb{R}^{n \times 10}$ and optional $u \in [0, 1]^n$, the generator computes $y_{\text{clean}} \in \{0, 1\}^n$.

10.1 Linear

$$p_i = \sigma(\mathbf{c}^\top \mathbf{X}_{i,C}), \quad y_{\text{clean},i} = \begin{cases} \mathbf{1}[u_i < p_i] & \text{data generation (Bernoulli)} \\ \mathbf{1}[p_i > 0.5] & \text{probes (MAP)} \end{cases}$$

Lemma 10.1 (Monotone directions). *$P(y_{\text{clean}} = 1 \mid x) = \sigma(\mathbf{c}^\top x_C)$ is strictly increasing in x_j when $c_j > 0$ and strictly decreasing when $c_j < 0$, for every value of the other features. The direction of feature j is therefore well defined: $s_j = \text{sign}(c_j)$.*

Proof. $\partial p / \partial x_j = \sigma'(\mathbf{c}^\top x_C) c_j$ and $\sigma' > 0$ everywhere. $\square$

Reliability of the causal signal. With $|C| = 3$ and $|c_j| \sim \text{U}(1, 2)$, the Bayes accuracy (the accuracy of the Bayes classifier; Devroye et al., 1996) from the load-bearing features, $\mathbb{E}[\max(p, 1 - p)]$, has mean 0.817 (5th–95th percentile 0.788–0.840 over tasks, by simulation). This is comparable to the spurious strength $s \in [0.80, 0.90]$, so in linear tasks the spurious feature is about as reliable as the causal signal: the shortcut competes. The design follows Colored MNIST (Arjovsky et al., 2019), where flipping 25% of the labels caps the causal (digit-shape) signal at 75% accuracy while colour agrees with the label 80–90% of the time in training and 10% at test. Labels are therefore drawn from p_i rather than thresholded: a noise-free linear rule would be perfectly reliable and would dominate the shortcut.

10.2 Tree (signed majority)

With $k = 3$ load-bearing features C_1, C_2, C_3 and bits $b_j = \mathbf{1}[X_{i,C_j} > 0]$, the leaf index is $L = b_1 + 2b_2 + 4b_3$ and $y_{\text{clean}} = \text{leaf_labels}[L]$. Splitting at zero follows the decision-tree class of Garg et al. (2022), whose depth-4 trees send an input left or right by the sign of a randomly chosen coordinate; here the tree has depth 3 with one feature per level, and its binary leaf labels are constrained as follows. Leaf tables are sampled by `_sample_tree_leaf_labels`, which imposes complement antisymmetry, $\text{leaf_labels}[L] = 1 - \text{leaf_labels}[7 - L]$, and rejects dictator and parity functions.

Theorem 10.2 (Admissible trees are signed majorities). *For $k = 3$, the leaf tables accepted by `_sample_tree_leaf_labels` are exactly the 8 functions*

$$y_{\text{clean}} = \text{Maj}(\ell_1, \ell_2, \ell_3), \quad \ell_j \in \{b_j, 1 - b_j\}.$$

Proof. Complement antisymmetry says $f(\bar{b}) = 1 - f(b)$, where $\bar{b}$ flips all three bits: f is *self-dual* (an odd function in the ± 1 notation of O’Donnell, 2014, which also defines the dictator, parity and majority functions used below). The 8 inputs form 4 complementary pairs $\{b, \bar{b}\}$, and a self-dual function is determined by a free choice of its value on one member of each pair, so there are exactly $2^4 = 16$ self-dual functions of 3 bits.

The sampler rejects the 6 dictators b_j and $1 - b_j$ (each self-dual) and the 2 parities $b_1 \oplus b_2 \oplus b_3$ and its complement (self-dual, because flipping three bits flips the parity). That leaves $16 - 8 = 8$ functions.

Each signed majority is self-dual, since $\text{Maj}(1 - \ell_1, 1 - \ell_2, 1 - \ell_3) = 1 - \text{Maj}(\ell_1, \ell_2, \ell_3)$. It depends on all three bits, so it is not a dictator. It is monotone in each literal while parity is not, so it is not a parity. There are $2^3 = 8$ sign patterns, giving 8 distinct functions (distinctness is checked by exhaustive enumeration in Appendix B). These 8 functions lie in the admissible set of size 8, so they are the whole set. $\square$

Corollary 10.3 (Tree directions). *Every admissible tree task is monotone in the sign of each load-bearing feature, with direction $s_j = +1$ if $\ell_j = b_j$ and $s_j = -1$ if $\ell_j = 1 - b_j$. Tree rules are deterministic, so the Bayes accuracy from the load-bearing features is $1 - \eta = 0.98 > s$: in tree tasks the causal rule dominates the shortcut. Linear and tree tasks therefore contrast a competitive shortcut with a dominated one.*

A dominated shortcut can still be adopted: Nagarajan et al. (2021) show that gradient-trained classifiers rely on a spurious feature more as its correlation grows, even when the invariant features predict the label perfectly. Tree tasks therefore still test reliance on f_8 , with a weaker incentive.

Regime. `_regime` encodes the sign pattern of the load-bearing features (`load_bearing_features()`: all causal features for linear tasks, the first 3 for tree tasks) with the leaf-index encoding $b_1 + 2b_2 + 4b_3$. For a tree task it therefore equals the leaf (tested); for linear tasks it is the orphant, used only for stratification. (In v2 it used all 3–5 causal features while `tree` used only the first 3, splitting each true leaf.)

Directions. `directions()` returns s_j for each load-bearing feature: $\text{sign}(c_j)$ for linear tasks, and for tree tasks the sign of the label difference between $b_j = 1$ and $b_j = 0$ over the leaf table, which recovers the literal signs of the signed majority. A test nudges every load-bearing feature by $+s_j$ and checks that the share of rows with the MAP label 1 rises.

10.3 Families excluded from evaluation

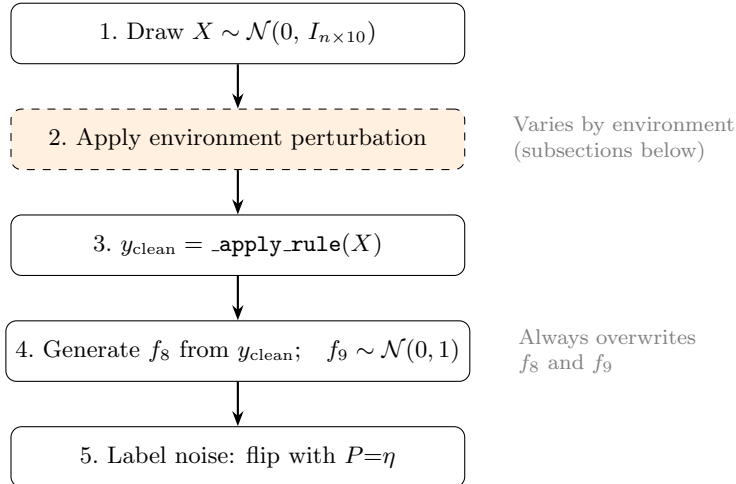
Lemma 10.4 (Threshold confounds naming). *The threshold rule $y = \mathbf{1}[\sum_j \mathbf{1}[x_{C_j} > t_j] \geq 2]$ is non-decreasing in every load-bearing feature, so $s_j = +1$ for all j .*

Under the naming protocol (Section 16), aligned threshold tasks would always carry positive-association names and flipped ones negative-association names, so the naming condition would be confounded with name valence. Sparse interaction, $y = \mathbf{1}[x_i x_j > \theta]$, is not monotone in x_i (the sign of its effect depends on x_j), so no direction exists. Both remain in the generator but are not evaluated.

11 Environment generation

Status: Implemented (P1); covariate shift redesigned in P2

Every call to `generate_environment()` follows the same skeleton:



11.1 id — in distribution

No perturbation.

11.2 covariate — $P(X)$ shift

Covariate shift changes $P(X)$ while the labelling rule $P(y | x)$ stays fixed (Shimodaira, 2000; Moreno-Torres et al., 2012). Transformers trained to learn linear functions in context are brittle under it (Zhang et al., 2024), which motivates testing it here. The shift should move the rule’s own inputs, so that queries fall where the demonstrations are sparse, without changing the label rate: a change of $P(y)$ would add prior-probability (label) shift (Moreno-Torres et al., 2012).

The shift vector moves three features: two load-bearing features in opposite label directions and one distractor. Draw two distinct load-bearing features i, j at random, with directions s_i, s_j , and a

distractor d from the features in $\{0, \dots, 7\}$ outside C (features 8 and 9 are overwritten in step 4, so shifting them would do nothing). With a push m ,

$$\delta_i = s_i m / |c_i|, \quad \delta_j = -s_j m / |c_j|, \quad \delta_d \sim U(1, 2) \times \text{Rademacher}.$$

For linear tasks $m \sim U(\max(|c_i|, |c_j|), 2 \min(|c_i|, |c_j|))$, which puts both $|\delta_i|$ and $|\delta_j|$ in $[1, 2]$ (the interval is never empty, because $|c| \in [1, 2]$). Tree rules read only signs, so $|c_i| = |c_j| = 1$ and $m \sim U(1, 2)$. Every shifted feature therefore moves by 1–2 standard deviations: feature i towards label 1, feature j towards label 0, and the distractor up or down at random.

Lemma 11.1 (The label rate is unchanged). (a) *Linear: $\mathbf{c}^\top(x + \delta) = \mathbf{c}^\top x$ for every row, so every row’s label distribution, and hence $P(y = 1)$, is unchanged.*

(b) *Tree: $P(y_{\text{clean}} = 1)$ stays $1/2$.*

Proof. (a) $c_i \delta_i = s_i |c_i| \cdot s_i m / |c_i| = m$ and $c_j \delta_j = -m$, and $d \notin C$ contributes nothing. (b) Let v_k be feature k ’s vote, $\mathbf{1}[s_k x_k > 0]$ (Section 10.2); the votes are independent, and each is 1 with probability $1/2$ before the shift. After it, $s_i(x_i + s_i m) > 0$ with probability $p_i = \Phi(m)$ and $s_j(x_j - s_j m) > 0$ with probability $p_j = 1 - \Phi(m)$; the third vote is unchanged. The majority is 1 when both shifted votes are 1, or exactly one of them is and the third vote is 1: $p_i p_j + \frac{1}{2}[p_i(1 - p_j) + (1 - p_i)p_j] = \frac{1}{2}(p_i + p_j) = \frac{1}{2}$. $\square$

The manifest records each shifted feature’s role (`towards_1`, `towards_0`, `distractor`), the push m , and a check (not a filter): the MAP label-1 rate of the same 20,000 fresh rows before and after the shift. Over the 24 evaluation tasks the difference is exactly 0 for the linear tasks and at most 0.009 for the tree tasks.

Why not rejection sampling. Until 30 September 2026 the shift moved 3 features drawn at random from $\{0, \dots, 7\}$ and was redrawn (up to 50 times) until the label-1 rate, estimated on a 2048-row probe, was within 0.03 of the ID rate. That filter selects distractors: a shift of distractors cannot change any label and always passed, while a shift of exactly one load-bearing feature moved the rate by a median of 0.23 and never passed. In the evaluation suite 21 of 24 tasks shifted distractors only, so for most tasks the environment tested sensitivity to irrelevant features rather than a shift of the rule’s inputs. The rejection sampler is kept only for the families that are not evaluated (threshold, sparse interaction). The redesign changed only the covariate rows: the pool, the ID and spurious-reversal rows and every query id outside the covariate environment are identical (checked file by file).

11.3 Covariate probes (exploratory, added after P4)

The pre-registered covariate contrast C1a found no harm (AUROC +0.024). Two properties of the `covariate` design explain much of this (`hiccups/18`). Every covariate row receives the same shift vector, so for a scorer roughly linear in the features the shift cannot reorder rows within the environment, which is what within-environment AUROC measures; and f_8 is regenerated from the label in every environment, so the shortcut stays valid. Qwen’s scores do move: for the same prompt, covariate queries score 0.44 to 1.21 logits higher than ID queries (with demonstrations only), and ranking ID positives against covariate negatives gives AUROC 0.46–0.54 under abstract names. Three probe splits, appended after the standard ones so that every existing row, value and query id is unchanged, test the covariate question further:

- `covariate_scale`: every covariate except f_8 is multiplied by its own factor $a_j \sim \mathcal{U}(1.5, 3)$, fixed per task, before the rule is applied. Each row moves in proportion to its own values, so rows can be reordered for a scorer whose weights differ from the rule’s, and values reach about 6 SD. $P(y | x)$ is unchanged, the label rate stays $1/2$ (every feature is symmetric), and f_8 is generated from the labels as usual.
- `<env>_f8neutral` for `id`, `covariate` and `covariate_scale`: the same queries with f_8 replaced by pool f_8 values drawn at random, so it no longer carries the label; query ids match the source, so the pairs are matched query by query.

On Qwen2.5-7B-Instruct (24 tasks, 3 seeds), the variance shift lowers AUROC only under flipped names (by 0.09–0.13; intervals above 0 for random, label_diversity and rule_diversity), where stretching the rule features amplifies a reading of them in the wrong direction. Making f_8 uninformative lowers ID AUROC by 0.05–0.08 (not under counter_spurious demonstrations, which never teach it), but does not widen the covariate gap (about 0), so shortcut reliance does not hide covariate harm here. These contrasts are exploratory and not part of the frozen family (`EXPLORATORY_CONTRASTS` in `src/evaluation/analysis.py`).

11.4 spurious_reversal

$s' = 1 - s$; the direction σ_t and everything else are unchanged. Sign agreement between $\sigma_t f_8$ and the label falls from s to $1 - s$ (Proposition 12.1). This mirrors the test environment of Colored MNIST (Arjovsky et al., 2019), where colour–label agreement falls from 80–90% in training to 10% while the causal mechanism is unchanged. Chen et al. (2026) use the same parameters to test transformers trained for in-context learning and find that they fail to adapt when the correlation reverses.

11.5 mechanism — why it is a placebo for in-context learning

The rule is applied to $\tilde{X}$ with $\tilde{X}_{:,C} = -X_{:,C}$, and for linear tasks the coefficients are rescaled, $c'_j = c_j v_j$ with $v_j \sim \text{U}(0.5, 1.5)$. The model sees the original X .

Proposition 11.2 (The mechanism shift inverts the ID rule). *Let $y_{\text{id}}(x)$ and $y_{\text{mech}}(x)$ be the clean labels of the same task in the two environments.*

- (a) *Tree: $y_{\text{mech}}(x) = 1 - y_{\text{id}}(x)$ for almost every x .*
- (b) *Linear: $P(y_{\text{mech}} = 1 \mid x) = 1 - \sigma(\mathbf{c}'^\top x_C)$, where $\mathbf{c}'$ has the same signs as $\mathbf{c}$. The Bayes decision of the mechanism environment, $\mathbf{1}[\mathbf{c}'^\top x_C < 0]$, is the negation of the ID Bayes decision, $\mathbf{1}[\mathbf{c}^\top x_C > 0]$, everywhere except on the set where $\text{sign}(\mathbf{c}'^\top x_C) \neq \text{sign}(\mathbf{c}^\top x_C)$, a region next to the ID decision boundary.*

Proof. (a) Negating the load-bearing features maps each bit to $1[-x_j > 0] = 1 - b_j$ (except on the null set $x_j = 0$), so the leaf index becomes $7 - L$, and complement antisymmetry gives $\text{leaf_labels}[7 - L] = 1 - \text{leaf_labels}[L]$. (b) $P(y_{\text{mech}} = 1 \mid x) = \sigma(\mathbf{c}'^\top (-x_C)) = 1 - \sigma(\mathbf{c}^\top x_C)$ because $\sigma(-z) = 1 - \sigma(z)$. The mechanism Bayes decision is therefore $\mathbf{1}[\mathbf{c}'^\top x_C < 0]$. Since $v_j > 0$, $\mathbf{c}'$ has the signs of $\mathbf{c}$, and wherever $\text{sign}(\mathbf{c}'^\top x_C) = \text{sign}(\mathbf{c}^\top x_C)$ this equals $1 - \mathbf{1}[\mathbf{c}^\top x_C > 0]$. $\square$

Corollary 11.3 (Consequences). (i) *Demonstrations from the ID pool. A model that recovers the ID rule from ID demonstrations is systematically wrong on mechanism queries (accuracy about $1 - \text{acc}_{\text{ID}}$). Only f_8 , generated from the post-shift label, keeps its relation to the label, so above-chance accuracy indicates spurious reliance rather than generalisation.*

- (ii) *Demonstrations from the mechanism environment. X is symmetric ($X \stackrel{d}{=} -X$), so the mechanism environment is distributed as another task of the same family with its signs flipped. With matched demonstrations it is just a new ID task, and $\text{ID} \approx \text{OOD}$ by construction: a placebo.*
- (iii) *Naming. If feature names are aligned with the ID rule, the mechanism environment with matched demonstrations is the ID environment with flipped names, up to the coefficient rescale (Proposition 16.4).*

Changing the rule while $P(X)$ stays fixed is concept shift (Moreno-Torres et al., 2012); in causal terms it replaces one mechanism and leaves the others unchanged (Peters et al., 2017). On real tabular data most large performance drops are dominated by such $Y \mid X$ shifts (Liu et al., 2023), so concept shift needs a valid operationalisation. It is therefore realised through naming (Section 16), an input-side analogue of the flipped labels of Wei et al. (2023). The mechanism environment is kept for SATA training, where it is always paired with matched demonstrations, as on `main`.

11.6 extrapolation and missing_feature

Extrapolation replaces 2–3 load-bearing features per row with $\pm \mathcal{U}(2, 4)$; missing feature redraws the first load-bearing feature independently of the label. Both are retained in the generator and not evaluated.

12 Spurious feature construction (f_8)

Status: Implemented (P1); per-task direction added in P2

$$d = \Phi^{-1}(s), \quad X_{i,8} = \sigma_t((2y_{\text{clean},i} - 1)d + \varepsilon_i), \quad \varepsilon_i \sim \mathcal{N}(0, 1),$$

where $\sigma_t \in \{+1, -1\}$ is the task’s spurious direction (Section 9). So $X_8 \mid y=1 \sim \mathcal{N}(\sigma_t d, 1)$ and $X_8 \mid y=0 \sim \mathcal{N}(-\sigma_t d, 1)$: in half of the tasks a large f_8 indicates label 1, in the other half label 0. This is a class-conditional Gaussian feature, like the environmental features in the data model of Rosenfeld et al. (2021), whose class-conditional mean and variance change across environments; here only the sign of the mean changes (spurious reversal). The range of s matches the colour–label agreement in the training environments of Colored MNIST (Arjovsky et al., 2019).

Proposition 12.1 (Sign agreement). $P(\text{sign}(\sigma_t X_8) = \text{sign}(2y_{\text{clean}} - 1)) = s$; under spurious reversal it is $1 - s$.

Proof. $\sigma_t X_8 = (2y_{\text{clean}} - 1)d + \varepsilon$. For $y = 1$: $P(\sigma_t X_8 > 0) = P(Z > -d) = \Phi(d) = s$. For $y = 0$: $P(\sigma_t X_8 < 0) = P(Z < d) = s$. Under reversal $d' = \Phi^{-1}(1 - s) = -d$, so the same computation gives $1 - s$. $\square$

Why the direction is random. In v2 and in the first P2 run, $\sigma_t = +1$ for every task. With abstract names, the zero-shot margin of Qwen2.5-7B-Instruct on the pilot tasks rose with every feature value: its Spearman correlation with the plain sum of the 10 features was 0.70, and 82% of the per-task least-squares slopes of the margin on the features were positive. The load-bearing directions are random, so such a prior helps in some tasks and hurts in others; the spurious direction was not, so the prior backed the shortcut in every task. Scoring the ID queries by the sum of the features gives AUROC 0.628, and 0.511 once f_8 is left out; Qwen’s zero-shot AUROC was 0.633, so its zero-shot signal came from f_8 through the prior. Under spurious reversal the same prior would have lowered every condition, zero-shot included, confounding demonstration-induced shortcut use with a prior shared by all strategies. With σ_t balanced, the prior backs the shortcut in half of the tasks and opposes it in the other half, and the two halves can be compared (prior-aligned against prior-opposed shortcut).

Because σ_t multiplies the whole column, $\sigma_t = -1$ changes nothing but the sign of f_8 : the other columns, the labels, $|f_8|$, the agreement flags and therefore the counter-spurious demonstration sets are identical to those of $\sigma_t = +1$ (tested). The archived first P2 run and the rerun therefore differ only in the sign of f_8 on the 6 pilot tasks with $\sigma_t = -1$.

Proposition 12.2 (Scale of f_8). With balanced classes, $\mathbb{E}X_8 = 0$ and $\text{Var} X_8 = 1 + d^2$. For $s = 0.80, 0.85, 0.90$ the standard deviation is 1.307, 1.440 and 1.626, against 1 for every other feature.

Proof. X_8 is an equal mixture of $\mathcal{N}(\pm d, 1)$; its mean is 0 and its second moment is $1 + d^2$. $\square$

Unstandardised, f_8 is visibly wider than the other columns (a v2 defect). v3 z-scores every feature with the task’s ID-pool mean and standard deviation.

Lemma 12.3 (What standardisation preserves). Z -scoring is an affine map $x \mapsto (x - \hat{\mu})/\hat{\sigma}$ with $\hat{\sigma} > 0$, so the Pearson correlation of f_8 with any variable is unchanged. Sign agreement is not preserved: centring $\sigma_t X_8$ at its pool mean $\hat{\mu}_8 = (2\hat{\pi} - 1)d$, where $\hat{\pi}$ is the pool label rate, changes agreement to $\hat{\pi} \Phi(d - \hat{\mu}_8) + (1 - \hat{\pi}) \Phi(d + \hat{\mu}_8)$.

Proof. $\text{corr}(aX + b, Y) = \text{corr}(X, Y)$ for $a > 0$. For the agreement, the displayed sign, read in the task’s direction, is $\text{sign}(\sigma_t X_8 - \hat{\mu}_8)$; condition on y and use the normal CDF. $\square$

At $s = 0.85$ and $\hat{\pi} \in \{0.35, 0.65\}$, agreement after centring is 0.817 against 0.850. Therefore every statistic about f_8 reliance is defined on the generator scale, and correlation-based statistics are preferred to sign-based ones.

13 Label noise and row metadata

Status: Implemented (P1)

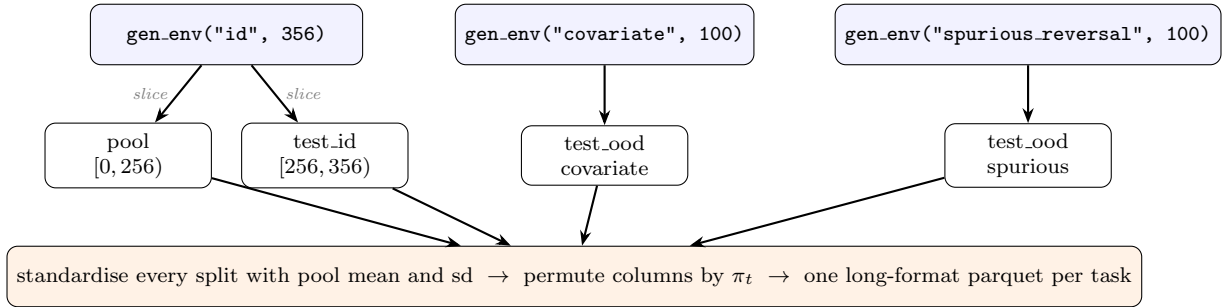
With probability $\eta = 0.02$ the label is flipped: $y = 1 - y_{\text{clean}}$ (random classification noise; [Angluin and Laird, 1988](#)). f_8 is generated from y_{clean} , so about 2% of rows have f_8 misaligned with the label the model sees. Each row stores:

Field	Definition
<code>y_clean, label</code>	rule label; label after noise (shown in prompts)
<code>regime</code>	sign pattern of the load-bearing features
<code>agree_clean</code>	$\mathbf{1}[\text{sign}(\sigma_t X_8) = \text{sign}(2y_{\text{clean}} - 1)]$ (generator scale)
<code>agree_obs</code>	$\mathbf{1}[\text{sign}(\sigma_t X_8) = \text{sign}(2y - 1)]$; used by counter-spurious selection
<code>spurious_raw</code>	X_8 on the generator scale, sign included (for magnitude matching)
<code>query_id</code>	0–19 for the environment’s queries, –1 otherwise

v2’s `is_counter_spurious` used y_{clean} ; it now follows the observed label (`is_counter_spurious` = $\neg$ `agree_obs`), because that is what the demonstrations show. The generator check of the spurious strength uses `agree_clean`, which the construction sets to s exactly.

14 Dataset construction

Status: Implemented (P1)



- Each task has its own pool. Demonstrations for a query from task t come only from task t ’s pool.
- ID rows are generated once per task and shared by all environments. This is deliberate: ID and OOD queries are then compared against the same demonstration sets (v2 produced identical ID data for each of its three datasets and triplicated the ID results).
- OOD splits are standardised with the *ID-pool* statistics, so a covariate shift appears as an offset of $\delta_j/\hat{\sigma}_j$ in z-units rather than being standardised away.
- Queries: 20 per task and environment, 10 per class, drawn with a fixed seed and numbered 0–19 in a random order, so a smaller run (the smoke run’s 4 queries) takes the lowest numbers of each class. Balanced query sets make accuracy equal to balanced accuracy and keep AUROC well defined.
- Every random choice of a task (ID rows, each OOD environment, the permutation, the queries) has its own seed, `SeedSequence([data_seed, purpose])`, so no draw depends on another.

- Files: `data/synthetic/v3/<suite>/tasks/<task_id>.parquet` and `data/synthetic/v3/<suite>/task_manifest.json`, written by `src/data/synthetic_bridge.py`. The parquet holds `task_id`, `split`, `env`, `row_id` (unique within the task), the 10 displayed columns, and the metadata above.
- `task_manifest.json` records every parameter needed to rebuild a task: family, load-bearing set, coefficients or leaf table, s , η , seeds, π_t , the column roles and directions under their displayed names, the spurious direction σ_t (`spurious_direction`), the standardisation statistics, domain, and the covariate-shift draw (features, δ , probe rates and difference, whether it was accepted). (v2's `task_meta.json` omitted coefficients, thresholds and leaf tables.)

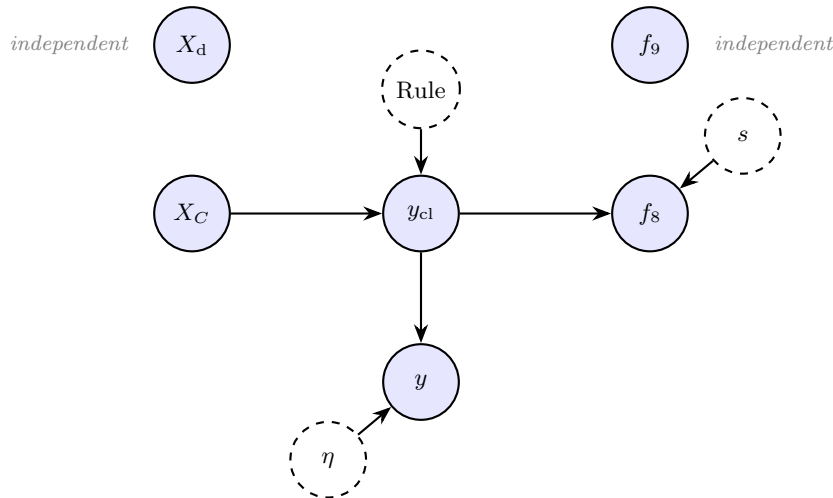
For the 24 evaluation tasks, every covariate shift moves two load-bearing features and one distractor, and the label rate is unchanged (probe difference 0 for linear and at most 0.009 for tree tasks). The spurious column lands in 7 of the 10 display positions, and the raw standard deviation of f_8 ranges from 1.29 to 1.61 before z-scoring (mean 1.45), against about 1.00 for the other columns.

Proposition 14.1 (No cross-task mixing). *Every demonstration pool used for a query from task t contains only rows from task t 's ID pool.*

Proof. Data are stored one task per file; the selection API receives one task's pool; the grid iterates over tasks, and a task's queries are scored only against demonstrations selected from that task's pool. $\square$

15 Causal graph

Status: Implemented



The graph follows the notation of causal directed acyclic graphs (Pearl, 2009). f_8 is a descendant of the label, not a cause: an anticausal feature in the sense of Schölkopf et al. (2012). Naming, serialisation and prompting are presentation layers and are not part of the data-generating process.

Part III

Presentation: naming, serialisation and prompts

16 Naming protocol

Status: Implemented (P3): `src/data/naming.py`, `configs/lexicons.yaml`

The naming factor carries the label manipulations of Wei et al. (2023) over to feature names. Abstract names play the role of their semantically unrelated labels, removing the prior; flipped names play the role of their flipped labels, making the prior contradict the rule. Nafar et al. (2025) compare named and anonymised features in in-context regression and find that names let models draw on prior knowledge in addition to learning from the examples, and Pan et al. (2023) separate task recognition, which relies on such priors, from task learning. Feature names matter for fine-tuned models too: correct names help and shuffled names hurt (Dinh et al., 2022). Language models’ knowledge of variable names can also serve directly as a prior (Choi et al., 2022).

Definition 16.1 (Lexicon). A domain D has a label meaning (1 and 0), a set of antonym pairs $\mathcal{P}_D = \{(p_m, q_m)\}$, where p_m is a name whose real-world association with label 1 is positive and q_m one whose association is negative, and a set $\mathcal{N}_D$ of plausible names with weak priors. Two domains are used: loan approval (1 = approved) and medical risk (1 = high risk). Domain is a between-task factor. The candidate and final lexicons are in Appendix A; the final names come from the P3 screening (Section 28).

Definition 16.2 (Namings). For task t with directions s_j ($j \in C$), assign each load-bearing feature a distinct pair $m(j)$ at random (seeded). Then:

- **abstract**: every column is named by its displayed position, f0–f9;
- **aligned**: feature j is named $p_{m(j)}$ if $s_j = +1$ and $q_{m(j)}$ if $s_j = -1$;
- **flipped**: the other member of the pair;
- in aligned and flipped, every other column (distractors, f_8, f_9) gets a distinct name from $\mathcal{N}_D$, identical in the two conditions.

Directions come from ground truth: $s_j = \text{sign}(c_j)$ for linear tasks (Lemma 10.1) and the literal signs for tree tasks (Theorem 10.2). The pool correlation of each feature with the label is computed only as a check. A task’s pairs and weak names are drawn from a stream seeded by the task alone (`naming_seed`), so the aligned and flipped namings share them.

The design intended pairs whose members have opposite measured prior slopes of similar magnitude, so that flipping changes the direction of the prior but not its strength. Qwen2.5-7B-Instruct’s priors are not symmetric (Section 28): a numeric prior makes almost every slope positive, and names that should lower $P(1)$ mostly lower the slope without reversing it. The final pairs are therefore those with the largest measured contrast $\beta_p - \beta_q$. Flipping then weakens the prior on positive-direction features and turns it against the rule on negative-direction features, where a positive name such as *credit score* sits on a column whose larger values lower the label. The effect is strong in the loan domain and weak in the medical domain.

Proposition 16.3 (Identification). *For a fixed task, demonstration set and query, the aligned and flipped prompts differ only in the names of the three load-bearing columns. The numbers, labels, column order and system message are identical. For any predictor whose output does not depend on display names, expected accuracy is identical under aligned and flipped naming, so a difference in the model’s accuracy between the two is attributable to the semantics of the names.*

Proof. By construction of the namings and the prompt template (Section 18); the demonstration sets of naming-blind selectors are identical across namings (Section 21). $\square$

Proposition 16.4 (Concept shift through naming). *Let names be aligned with the ID rule. With demonstrations drawn from the same environment, the mechanism environment presents the model with a rule whose directions are the negations of the names’ directions, which is exactly the flipped condition on ID data (up to the linear coefficient rescale).*

Proof. By Proposition 11.2, the mechanism rule has directions $-s_j$. Aligned names encode $+s_j$, so names and rule disagree on every load-bearing feature, which is the definition of the flipped condition. $\square$

What the names reveal. In named conditions the load-bearing columns carry strong-prior names and the others weak-prior names, so name salience correlates with causal relevance by design: the experiment manipulates the *direction* of the prior, not which features it points to. The LLM and SATA-MI can both use this; the SATA-names control (Section 36) receives the same information, so comparisons between them are fair. An optional robustness check gives one distractor a strong-prior name.

17 Serialisation

Status: Implemented (P1)

Serialising rows as text for a language model follows TabLLM (Hegselmann et al., 2023), which compares nine serialisations including a list of name–value pairs, and LIFT (Dinh et al., 2022). The format here is a compact name–value list. Each row is converted to text by `serialise_row()`:

```
f0: 0.80; f1: -0.45; f2: -0.30; f3: 1.52; f4: -0.91; f5: 1.10; f6: 0.07; f7: -1.33; f8:
  0.44; f9: 0.22 -> 1
credit score: 0.80; branch number: -0.45; ... ; debt-to-income ratio: 1.10; ... -> 1
```

- Format `name: value; ... -> label`; query rows end with `->` (instruct models) or `->` (base models, Section 18).
- Values are z-scores, fixed width with 2 decimals (`f"{v:.2f}"`).
- Column order is the displayed order after π_t , frozen per task and identical in every condition. `ordered_feature_names` sorts by raw column name, and the raw names are stored in displayed order, so renaming never reorders a prompt.
- Display names come from the naming’s codebook (`name_extended`).

18 Prompt templates

Status: Implemented (P1): abstract condition; the named system messages are used from P3

The v2 template rendered “Given the features of an individual, predict Classify the input into one of two classes...”, which is ungrammatical and a leftover from the real-data arm. v3 uses one template (`src/inference/prompts.py`); snapshot tests freeze the rendered prompt for both instruct models and the base format (`tests/snapshots/`).

System message (abstract).

```
You are a classifier. Each example lists 10 numeric measurements and a label (0 or 1). All
measurements are standardised (0 = average, 1 = one standard deviation above average).
The label follows an unknown rule over some of the measurements. Infer the rule from
the labelled examples, then classify the final example. Respond with exactly one
character: 0 or 1.
```

System message (named, loan domain).

```
You are a classifier. Each example describes a loan applicant with 10 measurements and a
label: 1 means the loan was approved, 0 means it was denied. All measurements are
standardised (0 = average, 1 = one standard deviation above average). The label follows
an unknown rule over some of the measurements. Infer the rule from the labelled
examples, then classify the final example. Respond with exactly one character: 0 or 1.
```

In the medical domain the second sentence reads “Each example describes a patient with 10 measurements and a label: 1 means the patient is at high risk, 0 means low risk.” The system message depends only on the domain, so the aligned and flipped system messages are byte-identical. The user message is the newline-joined demonstrations followed by the query; zero-shot prompts contain only the query.

Instruct models. The (system, user) pair is rendered with the tokenizer’s chat template and the assistant generation prompt, so the next token should be the answer. P2 checks this directly, from the probability the model puts on the two label tokens (Section 24): Qwen2.5-7B-Instruct puts 0.996 there, Llama-3.1-8B-Instruct about 0.02. The rendered prompt below is Qwen2.5-7B-Instruct’s for task `eval_0000`, with its first two pool rows as demonstrations and its first ID query; it is the output of `render_prompt` for that row, copied from the snapshot (`tests/snapshots/prompt_qwen_instruct.txt`). Llama-3.1’s template adds its own header (a BOS token and a dated system preamble) around the same messages.

```
<|im_start|>system
You are a classifier. Each example lists 10 numeric measurements and a label (0 or 1). All
measurements are standardised (0 = average, 1 = one standard deviation above average).
The label follows an unknown rule over some of the measurements. Infer the rule from
the labelled examples, then classify the final example. Respond with exactly one
character: 0 or 1.<|im_end|>
<|im_start|>user
f0: 0.99; f1: 1.42; f2: 0.72; f3: -0.23; f4: -0.52; f5: 1.09; f6: 0.35; f7: -0.74; f8:
-0.99; f9: -1.63 -> 0
f0: -1.78; f1: 0.26; f2: -0.00; f3: 0.82; f4: -0.80; f5: 1.31; f6: -0.02; f7: 0.22; f8:
-0.19; f9: 0.99 -> 1
f0: 1.25; f1: 0.19; f2: 0.10; f3: -0.04; f4: -0.83; f5: 0.20; f6: 0.01; f7: 0.36; f8:
-0.31; f9: 0.65 -><|im_end|>
<|im_start|>assistant
```

In this task the load-bearing features are shown as `f3`, `f9` and `f7`, the spurious feature as `f4` and the noise feature as `f8`.

Base model. Qwen2.5-7B base uses a raw completion prompt: system text, blank line, demonstrations, then the query ending in `"-> "` (with a trailing space), because the tokenizers split `" 0"` into two tokens.

Tokenisation fact (verified)	Qwen2.5	Llama-3.1
"0", "1"	single tokens 15, 16	single tokens 15, 16
" 0"	[220, 15]	[220, 15]
beginning-of-sequence token	none	128000, inserted by the chat template

Templated strings are tokenised with `add_special_tokens=False`, so Llama prompts begin with exactly one BOS. (v2’s `MLXRunner` produced [128000, 128000, ...].)

Part IV

Demonstration selection

19 Selection engine

Status: Implemented (P1, P3); SATA in P5

Selection is a *mechanism* (which rows are preferred) crossed with a *composition* (how many of each label):

`select(mechanism, composition, pool, query, k, seed) → (pool indices, metadata).`

free: the mechanism decides the label counts. **balanced:** exactly $k/2$ demonstrations per class, the mechanism choosing within each class.

Proposition 19.1 (Label counts are controlled). *Under balanced composition every mechanism produces the same demonstration label multiset, $k/2$ zeros and $k/2$ ones, whenever each class has at least $k/2$ pool rows. Differences between balanced mechanisms therefore cannot be caused by label proportions, which are known to drive a model’s output distribution (majority-label bias; Zhao et al., 2021; Min et al., 2022).*

Proof. The quota fixes the count per class before the mechanism chooses rows. With 256 pool rows and a label rate in $[0.35, 0.65]$ (G1 gate), each class has at least about 90 rows, far more than $k/2 \leq 16$. $\square$

Strategy	Mechanism, composition	Per query	Uses
zero_shot	—	—	—
random	random, free	no	—
label_diversity	random, balanced	no	—
feature_range	feature_coverage, balanced	no	MI top-3 features of the pool
rule_diversity	gt_regime, balanced	no	ground-truth regime
counter_spurious	gt_counter_spurious (magnitude-matched), balanced	no	ground-truth agree_obs
similarity	feature_knn, balanced	yes	z-scored features
counter_prior	counter_prior, balanced	no	measured zero-shot prior (P3)
counter_prior_ matched	counter_prior in label_diversity’s (label, f_8) cells	no	prior, agree_obs (exploratory)
SATA-v2, SATA-MI	learned scores, balanced top- k	yes	Part VII

The label, feature-range, rule and counter-spurious strategies are the four diversity protocols proposed in the project’s literature review. Similarity selection follows the nearest-neighbour retrieval of Liu et al. (2022), and covering the relevant structure of a task echoes diverse and coverage-based selection (Levy et al., 2023; Gupta et al., 2023).

20 Mechanisms

Status: Implemented (P1, P3)

random. Uniform without replacement.

feature_coverage. The 3 pool features with the highest mutual information with the label are cut into 3 quantile bins, and a row’s cell is its combination of bins (up to 27 cells). Within each class, cells are visited in random order, one row each, until the quota is filled. Mutual information is estimated with scikit-learn’s nearest-neighbour estimator (Pedregosa et al., 2011; Kraskov et al., 2004; Ross, 2014). The version ported from `main` assigned rows feature by feature: the top feature’s three bins

absorbed every row, so the other two features never formed a stratum. On all 24 evaluation tasks the top-3 features include the spurious feature (with two rule features in 23 of them, and never the noise feature). With $s \in [0.80, 0.90]$ it is more predictive than any single rule feature (its AUROC alone is $\Phi(\sqrt{2}d) \approx 0.93$, Section 12), and the pool’s mutual information cannot tell it apart from a causal one. Covering its bins within each class over-samples the rows on which it disagrees with the label, so `feature_range`’s sets show it agreeing in 0.66 of rows, against 0.84 for `label_diversity` (`hiccups/19`).

gt_regime. Rows are grouped by ground-truth regime; within each class, regimes are visited in random order, one row each. A row is eligible for its class only if it is not a label-noise row ($y = y_{\text{clean}}$) and its class is the pool majority in its regime. Without that filter, the few noise rows sitting in the other class’s leaves would each form a stratum and be over-sampled. Ineligible rows are used only if the eligible ones run out. For a tree task each class occupies 4 of the 8 regimes, so with $k = 8$ every leaf is represented once, with its own label (tested on all 12 tree tasks).

feature_knn. Euclidean distance on the 10 z-scored columns; the $k/2$ nearest rows of each class. This is the nearest-neighbour retrieval of Liu et al. (2022), in feature space rather than sentence-embedding space. It depends only on numbers, so it selects the same demonstrations under every naming. It replaces v2’s text similarity with the sentence-transformers model `all-MiniLM-L6-v2` (Reimers and Gurevych, 2019; Wang et al., 2020), whose embeddings would change with the display names and confound the naming factor.

counter_prior. Score $\mathbf{1}[\text{sign } \tilde{m}_0(x_i; N) \neq 2y_i - 1] \cdot \mathbf{1}[y_i = y_{\text{clean},i}]$, where $\tilde{m}_0$ is the model’s measured zero-shot margin on pool row i under the unit’s naming, centred on the pool median (Section 28): rows whose label contradicts the model’s prior *and* follows the rule (so label-noise rows are not rewarded). Rows scoring 1 are taken in random order within each class. The strategy is active only when the task’s prior–data conflict exceeds 1/2; otherwise it shows `label_diversity`’s demonstration set (the same rows and order), so the two strategies coincide exactly. It is related to misconfidence-based selection (Xu and Zhang, 2024), which prefers demonstrations on which the model’s output departs from the true label; here the model’s own zero-shot answer on each candidate plays that role. The design used the surrogate’s prediction; the measured prior replaced it after the surrogate failed to fit (Section 28). The code keeps the surrogate as a fallback (`prior_slopes`).

counter_prior_matched (exploratory, added during P4). Qwen’s zero-shot answers rise with every value, including f_8 ’s, so where $\sigma_t = -1$ the rows that contradict the prior are mostly rows on which the shortcut is right: `counter_prior`’s sets held 91% shortcut-agreeing rows there, against 84% for `label_diversity` (`hiccups/16`). The matched variant takes `label_diversity`’s set for the same seed as a reference and keeps its number of rows in each (label, f_8 agrees) cell, so label counts and the shortcut’s apparent reliability match `label_diversity`’s exactly; within each cell it prefers clean prior-contradicting rows, then other clean rows, then label-noise rows. Where `counter_prior` is inactive it is `label_diversity`’s set. It is not part of the contrast family.

20.1 Counter-spurious selection must match on $|f_8|$

The goal is a demonstration set in which f_8 carries no information about the label, so the model is not taught the shortcut. When demonstrations are consistent with more than one feature, models show systematic feature preferences (Si et al., 2023) and exploit shortcuts in the prompt (Tang et al., 2023), and in-context learners trained on tasks with spurious features adopt them (Harutyunyan et al., 2026; Chen et al., 2026). Demonstrations that break a spurious association improve generalisation (Fan et al., 2024). Worst-group robustness is defined over (label, spurious attribute) groups (Sagawa et al., 2020), and simply balancing those groups is a competitive remedy (Idrissi et al., 2022). Balancing the four (label $\times$ agreement) cells is not enough, however. Throughout this subsection f_8 is read in the task’s direction, $\sigma_t f_8$ (Section 12); $|f_8|$ and the agreement flags do not depend on σ_t , so neither do the selected sets.

Proposition 20.1 (Naive balancing leaves f_8 informative). *Take equal numbers of rows from the four cells (y , agree). Then sign agreement is exactly $1/2$, but*

$$\text{Cov}(f_8, 2y - 1) = \frac{1}{2} \left(\underbrace{d + \frac{\phi(d)}{\Phi(d)}}_{\mathbb{E}[|f_8| \mid \text{agree}]} - \underbrace{\frac{\phi(d)}{1-\Phi(d)} - d}_{\mathbb{E}[|f_8| \mid \text{disagree}]} \right) > 0.$$

Proof. Within class $y = 1$, $f_8 \sim \mathcal{N}(d, 1)$. Agreeing rows are $f_8 > 0$, with $\mathbb{E}[f_8 \mid f_8 > 0] = d + \phi(d)/\Phi(d)$; disagreeing rows are $f_8 < 0$, with $\mathbb{E}[f_8 \mid f_8 < 0] = d - \phi(d)/(1 - \Phi(d))$. The product $f_8(2y - 1)$ equals $+|f_8|$ on agreeing rows and $-|f_8|$ on disagreeing rows; class $y = 0$ is symmetric. With equal cell counts, $\mathbb{E}[f_8] = 0$ and the covariance is the average of $f_8(2y - 1)$, which gives the expression. It is positive because agreeing rows lie further from zero. $\square$

s	d	$\text{sd}(f_8)$	$\mathbb{E}[f_8 \mid \text{agree}]$	$\mathbb{E}[f_8 \mid \text{disagree}]$	naive corr(f_8, y)	zero-cov. counter share
0.80	0.842	1.307	1.192	0.558	0.281	0.681
0.85	1.036	1.440	1.311	0.518	0.334	0.717
0.90	1.282	1.626	1.477	0.473	0.391	0.757

A simulation at $s = 0.85$ gives $\text{corr} = 0.332$ for naive balancing. The v2 rule ($k/2 + 1$ counter-spurious rows, a share of 0.625 at $k = 8$) gives sign agreement 0.375, reversing f_8 's sign relation and mirroring the spurious-reversal test, while the covariance stays positive (0.168).

Definition 20.2 (Magnitude-matched counter-spurious). Within each class, draw $k/4$ disagreeing rows at random; for each, take the agreeing row of the same class with the nearest $|f_8|$, without replacement (nearest-neighbour matching on $|f_8|$; Rubin, 1973; Stuart, 2010), with $|f_8|$ on the generator scale. This requires k divisible by 4, which holds for $k \in \{8, 16, 32\}$. Label-noise rows ($y \neq y_{\text{clean}}$) are excluded on both sides: they are mostly disagreeing rows (a flipped label makes an agreeing f_8 disagree), so leaving them in would give counter-spurious sets several times the pool's noise rate. If a class has fewer than $k/4$ eligible disagreeing rows, the remaining slots are filled with random rows of that class and the shortfall is recorded.

Proposition 20.3. *With perfect matching, sign agreement is $1/2$ and $\text{Cov}(f_8, 2y - 1) = 0$.*

Proof. Within each class, agreeing and disagreeing rows then have the same distribution of $|f_8|$, so $f_8(2y - 1)$, which is $+|f_8|$ or $-|f_8|$ with equal frequency, has mean 0; $\mathbb{E}f_8 = 0$ by class symmetry. $\square$

With a 256-row pool matching is close to perfect; the simulation gives $\text{corr} = 0.000$ and agreement 0.500. The G1 gate checks that the correlation, pooled over counter-spurious demonstration sets, is within ± 0.1 of zero. On the 24 evaluation tasks, pooled over $k = 8, 16, 32$, the implementation gives $\text{corr}(f_8, y) = 0.005$ and agreement exactly 0.500, with no shortfall.

21 Seeding, reuse and pairing

Status: Implemented (P1)

Every random choice uses its own NumPy (Harris et al., 2020) stream, derived as

```
SeedSequence([base, task, strategy, seed, purpose, ...])
```

(demo_seed in src/selection/protocols.py), with 3 demo seeds per task and strategy. Task and strategy names enter through CRC-32 hashes, so no seed depends on list order or on Python's per-process hash salt. **purpose** separates the selection draw, the order shuffle and the label shuffle of the P2 control; a query-conditional strategy also passes the query's row id. The seed never includes the model, the naming, the label mode or the environment, so those comparisons see the same demonstration rows.

- **Query-agnostic strategies** draw one demonstration set per (task, strategy, seed). It is reused for every query of that task, in every environment, and under every naming for naming-blind strategies. `counter_prior` depends on the model and the naming (through the measured prior), so it is reused across environments only; where it is inactive it shows `label_diversity`’s set.
- **Query-conditional strategies** (`feature_knn`, `SATA`) select per query.
- **Order**: a seeded shuffle per demonstration set, so position is a controlled nuisance; order alone can swing in-context accuracy (Lu et al., 2022).
- **Shuffled labels (P2)**: the same rows in the same order, with their labels permuted within the set, which keeps the label counts and breaks the input–label mapping (Min et al., 2022).

This gives three pairings used by the analysis: the same queries across strategies; the same demonstration set across environments (the shift gap of a query-agnostic strategy is a within-prompt contrast, free of demonstration-sampling variance); and the same demonstration set across namings for naming-blind strategies. v2 passed one fixed seed to every call, so each strategy used a single demonstration set for all queries.

22 Pool provenance and leakage

Status: Implemented (P1)

Demonstrations always come from the task’s ID pool. Covariate and spurious-reversal queries are therefore answered with ID demonstrations, the deployment-realistic setting; concept shift is the flipped naming on ID data (Proposition 16.4).

Proposition 22.1 (No test leakage). *No row of `test_id` or of any `test_ood` split reaches any selector, standardisation statistic or prior estimate.*

Proof. Selectors receive only the pool; standardisation statistics are computed on the pool; prior surrogates are fitted on synthetic random profiles, and the measured priors are zero-shot answers on pool rows (Section 28); `SATA` is trained on separate training tasks. $\square$

v2 violated this: its counter-spurious proxy search used `test_ood` rows and labels.

23 Demonstrations without the shortcut (exploratory, added 5 October 2026)

Status: Implemented and tested (`src/data/demo_mask.py`); run pending (stage E; notebook 03.1); hiccups/20

Motivation. Demonstrations are examples meant to point the model to the signal that causes the label. The user’s argument is that they should therefore not show the shortcut feature, or the noise feature. The P4 design shows every feature because, in real data, no one knows which feature is spurious. This experiment asks how much of the shortcut problem disappears when the demonstrations never show the shortcut, while a query, as a new case would, still does. It does not replace the P4 design, and it is not part of the frozen contrast family.

Definition 23.1 (Demo masks). A mask $M \subseteq \{\text{spurious}, \text{noise}\}$ hides features from every demonstration line. Three arms of the grid are run: **none** ($M = \emptyset$, full demonstrations), **spurious** (Part 1) and **spurious_noise** (Part 2). The hidden displayed features are $f_{\pi_t(8)}$ for the shortcut and $f_{\pi_t(9)}$ for the noise feature: the generator’s columns 8 and 9 under the task’s recorded column-role permutation π_t (Section 8). They are looked up, never estimated, and checked against the roles in the manifest.

- **Queries are complete.** Every query and the content-free query show all ten features.
- **Selection is blind to hidden features.** The selection context holds the visible features only: `feature_range` ranks them by mutual information, and similarity measures distance on them. `counter_prior` reads Qwen’s zero-shot prior re-measured on the masked pool rows (Section 28).

- **Strategies that select on the shortcut** (`counter_spurious`, `counter_prior_matched`) are refused under a mask that hides it. `counter_spurious` runs in the full arm, where it neutralises the shortcut instead of removing it.
- **System message.** Masked demonstrations show 9 or 8 measurements and the query 10, so every arm uses the message without the count: “Each example lists numeric measurements and a label (0 or 1).” The full arm is therefore rerun rather than taken from P4.
- **Scope.** Qwen2.5-7B-Instruct, $k = 8$, the 24 evaluation tasks, three namings, three seeds (similarity one), and the id, covariate and spurious-reversal queries.

Proposition 23.2 (Same rows). *For random, label_diversity and rule_diversity, the demonstration rows and their order are the same in every arm. The prompts of the three arms then differ only in the hidden features’ name–value pairs on the demonstration lines and, against P4, in the system message’s count.*

Proof. Seeds never include the mask (Section 21). The random mechanism reads no feature, and `gt_regime` reads the regime, the label and the clean label only, so the selected rows and the seeded order do not depend on which features are visible. Queries are serialised from all ten features in every arm. $\square$

Analysis (`demo_mask_contrasts`). AUROC contrasts, paired by task, seed and query, with the hierarchical bootstrap, for every strategy, naming and environment:

- M1: no shortcut – full;
- M2: no shortcut or noise – full;
- M3: no shortcut or noise – no shortcut (what removing the noise feature adds);
- M4: the change in the ID – spurious-reversal gap, no shortcut – full;
- M5: label_diversity without the shortcut – `counter_spurious` with full demonstrations (removing against neutralising).

Expectations, stated before the run.

- If Qwen learns the shortcut from the demonstrations, hiding it lowers ID AUROC and raises spurious-reversal AUROC ($M4 < 0$).
- The query still shows the shortcut, and Qwen’s zero-shot habit is “bigger value, label 1” (Section 12). A gap that remains, with opposite signs for $\sigma_t = \pm 1$, is shortcut use through the prior.
- The noise feature is unrelated to the label, so M3 should be close to 0.

Compute. Stage E of `scripts/run_p4_pod.sh` on one H200, with 8 MPS workers: pool priors for the three arms ($3 \times 24 \times 3 \times 256$ zero-shot rows), then the three grids. It is about 310,000 forward passes. A one-task smoke run on the Mac rendered all three arms through Qwen’s chat template with label mass 1.0 and no prefix fallbacks.

Part V

Model inference

24 Models and runtime

Status: Implemented (P0–P1); measured in P2; Llama-3.1-8B-Instruct paused

Model	Layers	Width	Embeddings
Qwen2.5-7B-Instruct (primary)	28	3584	untied
Qwen2.5-7B base	28	3584	untied
Llama-3.1-8B-Instruct	32	4096	untied

Checkpoints (Qwen2.5: [Qwen et al., 2024](#); Llama 3.1: [Grattafiori et al., 2024](#)):

- Qwen/Qwen2.5-7B-Instruct
- Qwen/Qwen2.5-7B
- meta-llama/Llama-3.1-8B-Instruct (gated; access must be requested on Hugging Face)

PyTorch 2.14 ([Paszke et al., 2019](#)) and transformers 5.17 ([Wolf et al., 2020](#)), batch size 1, one model in memory at a time (15–16 GB each). Attention is `sdpa`; `eager` is used only if attention weights are needed.

Device selection (implemented, P0). `src/utils/device.py` chooses the device and dtype for every PyTorch component (the runner, SATA training and SATA selection):

- `resolve_device("auto")` returns NVIDIA CUDA if available, otherwise Apple Metal (MPS; [Apple Inc., 2026](#)), otherwise CPU; an explicit `cuda`, `mps` or `cpu` is honoured or fails loudly if unavailable. Configured by `inference.device` and `sata.device`.
- `resolve_dtype` keeps bf16 ([Kalamkar et al., 2019](#)) (verified on MPS) except on pre-Ampere CUDA GPUs such as the T4 and V100, which fall back to fp16, and on CPU, which uses fp32.
- On macOS, `PYTORCH_ENABLE_MPS_FALLBACK=1` is set before torch is imported, so an unsupported MPS operation runs on CPU instead of raising.

Reproducibility across devices. The device does not change the design, but CUDA, MPS and CPU use different kernels and summation orders, so logits can differ slightly (around the third decimal in bf16), enough to flip predictions that sit on the decision boundary. Every result row therefore records the device, its hardware name and the dtype, and each comparison is run on a single device. For SATA in fp32, scores on MPS and CPU agree within 10^{-5} (tested).

Single backend. Every model computation (scoring, prior measurement, SATA validation and the probes) runs in PyTorch through transformers. The v2 `MLXRunner` is removed in P1 once `HFRunner` passes its tests, so all results come from one implementation of tokenisation, caching and scoring. If PyTorch on MPS is slower than the compute budget assumes, the experiment is cut (the cut order in the research plan) rather than split across backends.

Answer format (P2). Scoring reads the two label tokens only, so it is meaningful only if the model intends to answer with one of them. The label mass $P(0) + P(1)$ at the answer position measures this. On the pilot, Qwen2.5-7B-Instruct’s median label mass was 0.996. Llama-3.1-8B-Instruct’s was 0.009–0.11 with demonstrations and 0.000 zero-shot: its most likely first token was To (probability 0.5–0.8), beginning “To infer the rule, I will analyze the given examples”, because the system message asks the model to infer the rule before it classifies. Its two-token scores then barely varied between queries (standard deviation 0.05 within a demonstration set, against 2.7 for Qwen), and its failed learnability gate (below) says nothing about learnability. A gate result therefore counts only if the model’s median label mass on the pilot is at least 0.9. Starting the assistant’s reply with `Label1:` raised Llama’s label

mass to about 0.99 on pilot prompts (both tokenizers split `Label: 1` into `Label, :, a space and 1`), but Llama was paused instead, by the user’s decision on 28 September 2026; the grids use the Qwen models for now.

25 Scoring

Status: Implemented (P1)

Predictions are read from the probabilities of the label tokens rather than from generated text, the usual way of scoring language models on classification tasks (Brown et al., 2020; Zhao et al., 2021). For each prompt, `HFRunner`:

1. tokenises the rendered prompt (no added special tokens);
2. runs the decoder once (without the output layer), so the output layer is applied at the last position only;
3. computes the logits of the tokens 0 and 1 in fp32 from the final normed hidden state h_L (two rows of the output layer), takes the full-vocabulary log-normaliser from the model’s output layer at the last position with those two logits substituted in fp32 (so the label probabilities never sum to more than 1), and forms the two-way probability as a logistic of the label-logit difference, which cannot underflow.

The model’s own bf16 output layer rounds its logits to bf16, a step of 0.125 at logit sizes near 20. In the first P2 run this put every margin on a 1/16 grid, left 19% of the p_1 values within a demonstration set exactly tied, and made cached and full passes disagree on near-ties (Llama: 96% of predictions, margin differences at most 0.25). Computing the two label logits in fp32 removes that rounding; everything before the output layer is unchanged. A test checks that the fp32 path equals the model’s own output when both run in fp32. The runner (`src/inference/hf_runner.py`) loads `.env` before touching the Hugging Face hub, because the gated Llama repository needs `HF_TOKEN` even when every file is cached.

Lemma 25.1 (Margin). *With $p_1 = e^{\ell_1} / (e^{\ell_0} + e^{\ell_1})$ (the constrained two-way softmax) and $m = \ell_1 - \ell_0$, we have $p_1 = \sigma(m)$. The margin is the same whether computed from logits or from full-vocabulary log-probabilities.*

Proof. $p_1 / (1 - p_1) = e^{\ell_1 - \ell_0}$. Log-probabilities are logits minus a common logsumexp, which cancels in the difference. $\square$

Prediction is $\hat{y} = \mathbf{1}[m > 0]$ and confidence is $\max(p_0, p_1)$. Because scoring is constrained to the two label tokens, no prediction is invalid.

26 Prefix caching

Status: Implemented (P1)

All queries that share a demonstration set share the prompt prefix up to the query line. The prefix is run once; each query suffix is scored against the cached keys and values, and the cache is cropped back to the prefix length afterwards. Key–value caching is standard in transformer inference (Pope et al., 2023), and reusing the cache of a shared prompt prefix is how serving systems avoid recomputing it (Gim et al., 2024; Zheng et al., 2024); the proposition below states when the reuse is exact.

Proposition 26.1 (Exactness). *For a decoder-only transformer with causal (masked) attention (Vaswani et al., 2017), the hidden states, and hence the cached keys and values, at positions $< |P|$ depend only on the tokens at those positions. Running prefix P once and then suffix S against the cache yields the same final-position logits as a full forward pass on $P||S$, provided that $\text{tok}(P||S) = \text{tok}(P)||\text{tok}(S)$ and the positions of S continue from $|P|$.*

Proof. By induction over layers. Embeddings, normalisation and MLPs act position by position; attention at position t reads keys and values at positions $\leq t$ only. So the states at positions $< |P|$ are

unchanged by appending S , and the states at the positions of S are computed from the same inputs in both cases. $\square$

Equality holds up to floating-point reduction order. The runner checks the token-prefix condition for every suffix, because a byte-pair tokenizer (Sennrich et al., 2016) can break it at the boundary; a suffix that fails the check is scored with a full forward pass instead, and the runner counts these fallbacks (none so far: prompts split at the start of the query line, which the tokenizers treat as a word boundary). After each suffix the cache is cropped back to the prefix length. Checks:

- G1 (passed): on the tiny test model, cached and full label log-probabilities agree within 10^{-4} (they are identical on CPU and MPS), and scoring the suffixes in reverse order gives the same values, so cropping leaves nothing behind;
- P2 (passed for Qwen2.5-7B-Instruct in bf16): caching moves the mean per-task AUROC by at most 0.005, a tenth of the learnability threshold; measured 0.0025, with 99.6% prediction agreement. The plan’s first rule, at least 99% prediction agreement, was replaced because a prediction can flip only on a near-tie, so its rate depends on the model’s margins rather than on the cache: all five Llama flips had $|m| \leq 0.022$.

27 Contextual calibration

Status: Implemented (P1)

Following Zhao et al. (2021), the model’s two-way output (c_0, c_1) on a content-free query (every value replaced by N/A) with the same demonstration prefix estimates its bias for that prompt. The calibrated probability is

$$\tilde{p}_1 = \frac{p_1/c_1}{p_1/c_1 + p_0/c_0}.$$

Lemma 27.1 (Calibration preserves ranks within a prompt). *For fixed (c_0, c_1) , $\tilde{p}_1$ is strictly increasing in p_1 . So within one demonstration set calibration does not change the ranking of queries, and AUROC computed within a demonstration set is unchanged; it changes thresholded metrics and AUROC pooled across demonstration sets.*

Proof. $\tilde{p}_1 = (1 + \frac{1-p_1}{p_1} \cdot \frac{c_1}{c_0})^{-1}$ and $(1-p_1)/p_1$ is strictly decreasing in p_1 . AUROC depends on the scores only through their ranking (Fawcett, 2006). $\square$

Zero-shot predictions are reported uncalibrated: with no demonstrations, the content-free prompt measures the prior this thesis studies.

28 Measuring priors

Status: Implemented (P3); G3 failed on the surrogate’s fit, naming kept with measured priors

Definition 28.1 (Prior surrogate). For a model and domain, the zero-shot margin $m_0(x; N)$ is measured on random profiles $x \sim \mathcal{N}(0, I_{10})$ in z-units, rounded to 2 decimals as in every prompt. Each profile names 3 columns from 3 distinct pairs (a random member of each) and the other 7 with distinct weak names, in random positions, as in a named task prompt, so that name and position are uncorrelated. Every model scores the same profiles. A ridge regression (Hoerl and Kennard, 1970) with $\alpha = 1$ fits

$$m_0(x; N) \approx \beta_0 + \sum_j \beta_{N(j)} x_j,$$

giving each name a prior slope β_{name} : its direction is $\text{sign } \beta$ and its strength $|\beta|$. Fit quality is the out-of-fold R^2 (5 folds); slope intervals come from 500 bootstrap resamples of the profiles. In the abstract condition the names are f0–f9 in their own positions.

The surrogate is a global surrogate model of the LLM’s zero-shot behaviour: an interpretable model fitted to a black box’s outputs on sampled inputs (Craven and Shavlik, 1996). LIME (Ribeiro et al., 2016) fits such models locally.

Screening (rules fixed before the run). 800 profiles per domain cover every candidate name (8 pairs and 10 weak names per domain, Appendix A); 400 profiles cover the abstract names. A pair (p, q) is admitted if $\beta_p > 0 > \beta_q$, both 95% intervals exclude 0, and $\min(|\beta_p|, |\beta_q|) / \max(|\beta_p|, |\beta_q|) \geq 1/2$; a weak name is admitted if $|\beta|$ is at most a quarter of the median $|\beta|$ of the admitted pair members. The code is `scripts/p3_prior_screen.py`.

Result (Qwen2.5-7B-Instruct).

Condition	Out-of-fold R^2	Intercept	Slopes
abstract	0.51	+0.00	all positive, 0.22 to 0.89
loan	0.20	−2.33	weak names +0.29 on average; 1 of 8 pairs has $\beta_q < 0$ with an interval below 0
medical	0.28	+1.41	weak names +0.48 on average; no negative slope below −0.05

- A numeric prior dominates: under any name, a larger value makes label 1 more likely. This is the prior found in P2 (Section 12).
- Names that should lower $P(1)$ mostly weaken the slope instead of reversing it. Only *missed payments* reverses clearly (−0.79); in the medical domain, *daily step count* (+0.74) and *HDL cholesterol* (+0.35) push towards high risk.
- The named fits are poor, and flexible models do not rescue them: gradient boosting over names, values and positions reaches 0.29 (loan) and 0.44 (medical).
- One pair of 16 passes the screening rule, so the rule cannot supply 3 pairs per domain.

Final names. By the user’s decision (30 September 2026), the 5 pairs per domain with the largest measured contrast $\beta_p - \beta_q$ and the 7 weak names with the smallest $|\beta|$ (Appendix A). In loan every kept contrast has an interval above 0 (the smallest is 1.34, [0.66, 2.17]). In medical only *systolic blood pressure / weekly exercise hours* is clear (1.62, [0.92, 2.32]); the other four lie between 0.29 and 0.67, and two intervals include 0.

Definition 28.2 (Measured prior). For task t , naming N and pool row i , the measured prior $\tilde{m}_0(x_i; N)$ is the model’s zero-shot margin on that row, under the same system message and names as the grid, minus its median over the pool.

Centring removes the model’s label bias (the intercept), as contextual calibration and AUROC remove it elsewhere, so the prior predicts label 1 for half of the pool. The measured prior is exact on the rows it is used on and costs 256 zero-shot queries per (task, naming) (`scripts/p3_pool_priors.py`, `PoolPrior`). It replaces the surrogate in `counter_prior` and in the conflict statistic below; the surrogate is kept as a description of the name priors.

Definition 28.3 (Prior–data conflict). For task t under naming N , with $\hat{y}_0(x) = \mathbf{1}[\tilde{m}_0(x; N) > 0]$,

$$C_t(N) = 1 - \frac{1}{|\text{pool}|} \sum_{i \in \text{pool}} \mathbf{1}[\hat{y}_0(x_i) = y_i].$$

Aligned tasks should have $C_t < 1/2$ and flipped tasks $C_t > 1/2$; this is checked on the evaluation tasks once the pool priors are measured (`results/v3/synthetic/p3/naming_gate.json`). The name follows prior–data conflict in Bayesian model checking (Evans and Moshonov, 2006). For language models, context that contradicts parametric knowledge is studied as knowledge conflict (Longpre et al., 2021; Xie et al., 2024); here the conflict is between the rule the names imply and the rule the demonstrations show.

Random-label control. Demonstrations with shuffled labels keep the label marginals and format but break the input–label mapping (Min et al., 2022), separating what the model takes from the mapping from what it takes from format and prior. In P4 it is run for `label_diversity` under every naming.

Gate G3. Pre-registered: out-of-fold $R^2 \geq 0.5$ in every domain, and zero-shot AUROC under aligned minus flipped naming ≥ 0.15 on the pilot’s ID queries (the names really carry priors). The draft stated the second criterion in accuracy; AUROC was fixed before the check ran, because zero-shot predictions are uncalibrated and raw accuracy mixes the model’s label bias into the contrast.

- R^2 : 0.20 (loan) and 0.28 (medical), so the criterion fails.
- Separation, on the 12 pilot tasks with the final names: zero-shot AUROC 0.499 abstract, 0.594 aligned and 0.430 flipped; the difference is 0.164, 95% interval $[0.053, 0.287]$, positive on 10 of 12 tasks (raw accuracy: 0.113). By domain it is 0.278 (loan) and 0.050 (medical).
- Decision (the user’s, 30 September 2026): keep the naming factor. `counter_prior` and C_t use the measured prior, so the surrogate’s fit no longer matters for any result; medical is reported as a weak-prior domain.

Part VI

Evaluation and statistics

29 Metrics

Status: Implemented (P1–P2): AUROC and balanced accuracy; the full analysis in P4

Metric	Definition
Accuracy	$\frac{1}{n} \sum_i \mathbf{1}[\hat{y}_i = y_i]$
Balanced accuracy (BA)	$\frac{1}{2}(\text{TPR} + \text{TNR})$; equal to accuracy on the 10/10 query sets
Calibrated BA (primary)	BA of $\mathbf{1}[\tilde{p}_1 > 1/2]$
AUROC (primary)	area under the ROC curve of p_1 , per (task, strategy, seed, environment) cell, averaged over cells
Macro-F1	mean of per-class F1
Prior-agreement rate (PA)	$\frac{1}{n} \sum_i \mathbf{1}[\hat{y}_i = \hat{y}_i^0]$, where $\hat{y}^0$ is the zero-shot prediction under the same naming
Shift gap	$\Delta = \text{BA}_{\text{ID}} - \text{BA}_{\text{OOD}}$
Demo-following index (DFI)	Section 31

For query-agnostic strategies, a cell’s AUROC is computed within one demonstration set and is therefore unchanged by calibration (Lemma 27.1). AUROC is the probability that a random positive query scores above a random negative one (Hanley and McNeil, 1982; Fawcett, 2006); balanced accuracy is the mean of the per-class accuracies (Brodersen et al., 2010); the shift gap is TableShift’s gap between ID and OOD accuracy (Gardner et al., 2023) with the sign reversed (TableShift reports OOD minus ID), so that a positive gap means degradation. Prior agreement and DFI are defined for this study.

30 Design and inference

Status: Hierarchical bootstrap implemented (P2); contrasts and Holm correction in P4

Units. Task (the unit of generalisation over rules), query within task, and demonstration seed. Per model, the full grid has $24 \text{ tasks} \times 3 \text{ namings} \times 3 \text{ environments} \times 20 \text{ queries}$, crossed with strategies and 3 seeds.

Why more tasks. For a per-task effect with between-task variance σ_T^2 and within-task variance σ_W^2 , the variance of the grand mean is $\sigma_T^2/T + \sigma_W^2/(Tn)$ (two-stage sampling; Cochran, 1977), so tasks, not queries, control precision when rules differ. v2 evaluated one task with $n = 30$ queries per cell (95% interval half-width 0.179 at accuracy 0.5).

Hierarchical bootstrap. For a contrast between conditions A and B on paired queries: (1) resample tasks with replacement; (2) within each resampled task, resample queries with replacement; (3) compute the metric difference on the resample; repeat $B = 2000$ times and report the percentile interval (Efron and Tibshirani, 1993) (`bootstrap_ci`, `paired_bootstrap_diff` from `main`). Resampling at both levels respects the nesting of queries within tasks (Davison and Hinkley, 1997; Saravanan et al., 2020).

Per-task sign tests count tasks on which A beats B (binomial sign test; Dixon and Mood, 1946), guarding against a few tasks driving the mean.

Multiplicity. Holm correction (Holm, 1979) over a pre-registered contrast family (Nosek et al., 2018), frozen on 30 September 2026 before any P4 result existed (CONTRASTS in `src/evaluation/analysis.py`). All ten contrasts are one-sided (the direction below is the alternative), paired by task and query, and tested with the hierarchical bootstrap (hierarchical_contrast, $B = 2000$); their p-values are Holm-adjusted together. The statistic for C1 and C2 is AUROC, computed within a cell from the calibrated p_1 (for a query-agnostic strategy this equals the AUROC of the raw p_1 ; for similarity it removes each query’s own prompt bias). Calibrated balanced accuracy is reported alongside and is not part of the family. The family was first frozen with calibrated balanced accuracy; on the same day, after the first 11 tasks (seed 0, abstract names) showed that contextual calibration often leaves every query of a plain-random cell on one side of the threshold (36% of cells), which pins balanced accuracy at 0.5 and forces those cells’ shift gaps to 0, the user chose AUROC, the study’s primary metric and the metric of the P2 and G3 gates (hiccups/15).

- C1. RQ1, random demonstrations. (a) ID minus covariate and (b) ID minus spurious reversal, both under abstract names, > 0 ; (c) aligned minus flipped names on ID queries (concept shift) > 0 .
- C2. RQ2. (a) counter_spurious minus label_diversity under spurious reversal; (b) similarity (feature_knn) and (c) feature_range minus label_diversity under covariate shift; (d) counter_prior minus label_diversity under flipped names on ID queries; (e) the interaction, (counter_spurious minus label_diversity) under spurious reversal minus the same under covariate shift. All > 0 ; (a)–(c) and (e) under abstract names.
- C3. RQ3. (a) DFI under abstract minus flipped names > 0 ; (b) $\rho(\pi_{\text{true}}, \pi_{\text{behav}})$ under abstract minus flipped names > 0 .
- C4. RQ4 (a separate family, fixed before P5): SATA-v2 – best hand-designed strategy ≥ 0 on worst-environment calibrated BA; SATA-MI – SATA-v2 > 0 and SATA-MI – SATA-names > 0 under flipped naming.

Two points were settled when the draft was frozen. Balanced strategies are compared with label_diversity (random, balanced), not with free random, so each RQ2 contrast compares mechanisms at fixed label counts (Proposition 19.1); plain random remains the RQ1 baseline. For a query-agnostic strategy an environment gap is a within-prompt contrast, and the bootstrap resamples the ID and OOD queries of each task separately.

31 Faithfulness and reliance (RQ3)

Status: Implemented (P4): `src/experiments/rq3.py`, `scripts/run_rq3.py`; analysis in `src/evaluation/analysis.py`

As in the project’s literature review, the design follows STaDS (Li et al., 2025), which measures decision faithfulness as the rank correlation between a model’s self-reported feature ranking and its ranking from leave-any-out ablation. Self-explanations can be unfaithful (Turpin et al., 2023; Lanham et al., 2023), and LLM-reported feature importance has previously been compared with occlusion and LIME (Huang et al., 2023).

- π_{true} : ground-truth importance from the generator (positive for load-bearing features, zero otherwise; `true_importance_scores` from `main`), available because the data are synthetic (Liu et al., 2021).
- π_{behav} : the mean drop in the correct-label margin $(2y - 1)(m - m_{\text{cf}})$ when feature j of the query is replaced by a donor row’s value (hot-deck imputation, Andridge and Little, 2010; `hot_deck_impute_feature`: one of the 5 nearest pool rows on the other 9 columns, at random). The drop in calibrated accuracy is reported alongside; with 20 queries it moves in steps of 0.05, and its ties make the rank correlation unstable, so the margin version is primary (fixed before P4). Drawing the replacement from the data keeps the perturbed query in distribution, the principle Zhu et al. (2026) argue for image attribution. It is removal by marginal sampling in the framework of Covert et al. (2021), and resembles permutation importance (Breiman, 2001).
- π_{self} : the model’s own ranking. After the same demonstrations, the ranking prompt asks: “Rank all 10 measurements from most to least important for predicting the label. Respond with the measurement names only, separated by commas, most important first.” Its system message is the grid’s without the answer-format sentence. The greedy answer is matched against the display

names (`parse_ranking`, which also accepts list markers and case changes); unlisted columns go last, and the number listed is recorded.

- Correctness $\rho(\pi_{\text{true}}, \pi_{\text{behav}})$ and internal consistency $\rho(\pi_{\text{self}}, \pi_{\text{behav}})$, Spearman with bootstrap intervals, by naming.

Definition 31.1 (Directional sensitivity and demo-following index). For query x and load-bearing feature j , let $g_j = m(x + \delta e_j) - m(x - \delta e_j)$ with $\delta = 0.5$ z-units. The response follows the data if $\text{sign } g_j = s_j$ and follows the name if $\text{sign } g_j$ equals the name’s lexicon direction (+1 for a pair’s first member, −1 for its second). The demo-following index is the fraction of (query, load-bearing feature) pairs with $\text{sign } g_j = s_j$.

The draft read the name’s direction from the surrogate, $\text{sign } \beta_{N(j)}$; with Qwen’s numeric prior almost every β is positive (Section 28), so that version would test “bigger values mean 1” rather than the name, and the lexicon direction is used instead.

Implementation. A unit is (model, naming, task, seed). It shows the grid’s label_diversity demonstration set for that task and seed (the same rows and order) and scores, against one cached prefix, every ID query, each query with each of its 10 columns replaced by its donor value, each query with each load-bearing column and f_8 moved by ± 0.5 (so reliance on the shortcut is also read, in the task’s direction σ_t), and the content-free query; 381 suffixes per unit. Donors depend only on the task and the column, so every naming and seed sees the same perturbed queries. The ranking prompt follows. Seed 0 runs first. Under flipped naming, following the data and following the name are mutually exclusive, so DFI measures, feature by feature, which side of the contest wins. It plays the role, for individual features, of the flipped-label accuracy of Wei et al. (2023).

Part VII

SATA

32 Architecture

Status: Planned (P5): ported from main

SATA was proposed in the project’s literature review. It draws on In-Context Risk Minimization (Gupta et al., 2024), in which a transformer reads other inputs from the test environment as context in order to adapt to that environment.

SATA scores each candidate demonstration given the whole pool and the query. Candidates are embedded as feature vector plus label embedding plus a demonstration-type embedding; the query is embedded as its feature vector plus a query-type embedding; a transformer encoder ($d = 128$, 4 heads, 4 layers) attends over all tokens; a linear head gives one score per candidate, normalised by a softmax. The query-agnostic ablation replaces the query with a zero vector. Selection takes the balanced top- k .

The row encoding resembles TabPFN (Hollmann et al., 2023), which also embeds each labelled row as one token and lets query rows attend to them. There are no positional encodings, so the encoder is permutation equivariant over candidates, the property that set-based attention models are built around (Lee et al., 2019).

Relation to learned selectors. Training a demonstration selector from language-model feedback is established: EPR (Rubin et al., 2022) and UDR (Li et al., 2023) train retrievers on the model’s scores of candidate demonstrations, CEIL (Ye et al., 2023) learns to select whole sets, and Zhang et al. (2022) learn a selection policy by reinforcement learning. Signals from model internals and learned judges of demonstration sets have also been proposed (Kato et al., 2025; Wang et al., 2026). SATA differs in being trained on synthetic tasks with ground-truth structural targets, in targeting robustness to controlled shift, and, in the MI variants, in using the model’s measured priors.

Variant	Per-row input	Width
SATA-v2	x (z-scored, displayed order)	10
SATA-names (control)	$[x, s^{\text{lex}} \odot x]$, lexicon directions ± 1 , 0 for neutral names	20
SATA-MI	$[x, \beta \odot x, \hat{m}_0(x)]$, measured prior slopes and surrogate margin	21

Since P3 the surrogate is descriptive only (its fit failed G3, Section 28); P5 may take $\hat{m}_0$ from the measured prior on the pool rows and the zero-shot margin of the query instead.

33 Training targets

Status: Planned (P5)

Structural targets (SATA-v2, as on main). For candidate i and query q ,

$$\tau_i = 2 \mathbf{1}[r_i = r_q] + 1.5 \mathbf{1}[\text{counter}_i] - 1.5 \mathbf{1}[\text{consistent}_i \wedge r_i \neq r_q], \quad q_i = \text{softmax}(\tau/T)_i,$$

where r is the regime and counter/consistent refer to **agree_obs**. Softening the targets with a temperature T follows knowledge distillation (Hinton et al., 2015).

Lemma 33.1 (No query-label term). *Every term of τ is a function of the candidate’s features and label and of the query’s features. None depends on the query’s label.*

Proof. The regime is a function of features; agreement is a function of the candidate’s f_8 and label. $\square$

(v1’s targets rewarded candidates whose label matched the query’s label, which, with a near-copy f_8 , taught SATA to read the query label from f_8 .)

SATA-MI. Adds $\lambda \mathbf{1}[\text{counter-prior}_i \wedge y_i = y_{\text{clean},i}]$ when $C_t(N) > 1/2$, rewarding candidates that contradict the prior and follow the rule when the prior is wrong.

SATA-MI-mech (datamodel utilities). For a training task and naming, sample $M = 128$ balanced subsets $S_1, \dots, S_M$ of size k from a 64-row candidate pool and measure, with the LLM,

$$U(S) = \frac{1}{|Q|} \sum_{q \in Q} (2y_q - 1) m(q; S),$$

the mean margin toward the correct label over a label-balanced query set Q drawn from the training task. Fit $U(S) \approx \theta_0 + \sum_i \theta_i \mathbf{1}[i \in S]$ by ridge regression, a linear datamodel (Ilyas et al., 2022), and use $q = \text{softmax}(\theta/T)$ as targets for the query-agnostic head. The closest precedent is Chang and Jia (2023), who fit linear datamodels to the correct-class margin of in-context examples to find examples that stabilise in-context learning; Nguyen and Wong (2023) estimate similar influences by comparing random subsets with and without each example. Here the fitted utilities become training targets for a selector.

Lemma 33.2 (No query-label leak). *θ is a property of the candidate pool: it depends on query labels only through an average over a label-balanced set Q from a training task. Evaluation queries never enter any target, so SATA cannot learn to infer an evaluation query’s label.*

Proof. Targets are computed on training tasks only, and the query-agnostic head receives no query input. $\square$

34 Training

Status: Planned (P5)

- **Tasks:** the evaluation distribution (Section 7) with disjoint ids; pool size 256, matching evaluation. Out-of-distribution in-context generalisation depends on the diversity of the training tasks (Goddard et al., 2025), so training covers the full task distribution rather than a few fixed tasks.
- **Environments:** id, covariate and spurious reversal (ID pool 75% of batches, matched pool 25%), and mechanism (always matched pool; Section 11.5).
- **Loss** $\text{KL}(q \parallel p_\theta)$; Adam (Kingma and Ba, 2015), learning rate 10^{-4} ; early stopping (Prechelt, 1998) on the proxy with patience 10.
- **Device:** auto-detected (`sata.device`); benchmarked once against CPU, because SATA’s batches are small and GPU launch overhead may outweigh the gain. A one-epoch run on MPS is verified.

35 Validation

Status: Planned (P5)

- **Oracle gate (S3a):** top- k by the target function itself (no model) must beat the hand-designed strategies on the proxy before any training.
- **XGBoost proxy:** an XGBoost classifier (Chen and Guestrin, 2016) is fitted on the selected demonstrations and predicts the query; worst-environment accuracy, the environment-level counterpart of worst-group accuracy (Sagawa et al., 2020), drives early stopping.
- **Prior-augmented proxy (MI variants):** $P(y = 1 \mid x) = \sigma(\alpha \hat{m}_0(x) + w^\top x)$, with w fitted by ridge on the k selected demonstrations and α on validation tasks. The XGBoost proxy cannot see names, so it cannot validate the model-informed component.
- **LLM validation:** 8 held-out validation tasks, all environments and namings, calibrated BA. Never evaluation tasks.
- **Gates:** 5b requires proxy-LLM Spearman ≥ 0.5 and SATA-MI $\geq$ both controls; 5c requires utility held-out $R^2 \geq 0.2$ and top- k by utility beating random under the LLM.

36 Leakage and fairness controls

Status: Planned (P5)

1. No target depends on an evaluation query’s label (lemmas above).
2. Column-role permutation removes the positional route to f_8 (Section 8).
3. Name salience reveals causal relevance by design; the SATA-names control receives the same information without any LLM measurement, isolating the value of the measured priors.
4. The counter_prior heuristic is the only other naming-aware selector, so SATA-MI is compared against a simple selector that sees the same prior.
5. Asymmetry, disclosed: rule_diversity and counter_spurious use ground-truth tags at inference, while SATA does not. This favours the hand-designed strategies.
6. Validation tasks are disjoint from evaluation tasks.

Part VIII

Mechanistic probes

37 Residual stream and logit lens

Status: Planned (P6)

The logit lens (nostalgebraist, 2020) reads a model’s intermediate predictions by applying the final normalisation and unembedding to each layer’s residual-stream state (Elhage et al., 2021). A forward pass with `output_hidden_states=True` returns $L + 1$ tensors: the embeddings and the output of each layer. In transformers 5.17 the last one has already been through the final norm (verified: applying `lm_head` to it reproduces the logits, applying the norm again does not). Let h_l be the last-position state and $w = W_U[1] - W_U[0]$ the difference of the two label rows of the (untied) unembedding.

Definition 37.1 (Logit-lens margin). $m_l = w^\top \text{RMSNorm}(h_l)$ for $l < L$, and $m_L = w^\top h_L$, where RMSNorm is the model’s final RMS normalisation layer (Zhang and Sennrich, 2019).

Lemma 37.2 (Final-layer identity). m_L equals the runner margin m of Lemma 25.1.

Proof. h_L is the normed final state and $\ell = W_U h_L$, so $m_L = \ell_1 - \ell_0 = m$. $\square$

The P6 gate checks this identity within 10^{-3} in bf16. It is attainable because the runner computes the label logits in fp32 from h_L , as the logit lens does (Section 25); margins read from the bf16 output layer would miss it by up to half a bf16 step, about 0.06.

38 Critical layer, probes and steering

Status: Planned (P6)

Definition 38.1 (Critical layer). Let $\text{acc}_l(N)$ be the accuracy of $\mathbf{1}[m_l > 0]$ against the rule label with demonstrations under naming N . The critical layer of a task is the smallest l at which $\text{acc}_l(\text{aligned}) - \text{acc}_l(\text{flipped})$ exceeds 0.1 with a bootstrap interval excluding zero: the point where prior and demonstration evidence diverge (after Halawi et al., 2024).

Linear probes. Per layer, logistic regressions on h_l decode the rule label and the prior prediction $\hat{y}^0$; the decodability curves show where each kind of information is linearly available (Alain and Bengio, 2017). Probes complement the logit lens, whose readings are biased across layers and unreliable for some model families (Belrose et al., 2023). Decodability shows that information is present, not that the model uses it (Belinkov, 2022), which is why steering serves as the causal check.

Steering (causal check). On training tasks, take the difference of mean states between prompts whose prediction follows the demonstrations and prompts whose prediction follows the prior:

$$v_l = \bar{h}_l(\text{follows demonstrations}) - \bar{h}_l(\text{follows prior}).$$

At the critical layer, set $h_l \leftarrow h_l + \alpha v_l$ with a forward hook and measure the change in accuracy on held-out tasks. Steering is an upper bound and a causal test of the diagnosis, not a selection method. Difference-of-means steering vectors follow activation addition (Turner et al., 2023) and contrastive activation addition (Rimsky et al., 2024). Minder et al. (2025) find a context-versus-prior direction of this kind in Llama-3.1-8B, and task and function vectors (Hendel et al., 2023; Todd et al., 2024) are the in-context-learning counterparts. Similar tools have been used to diagnose false demonstrations (Halawi et al., 2024).

Question for selection. Do counter_prior or SATA-MI demonstrations move the divergence point, or raise m_l toward the demonstrated label earlier, relative to random demonstrations? Kato et al. (2025) show that scores computed from induction-head representations track in-context accuracy, and leave selection with such scores to future work.

Part IX

Experiment orchestration

39 Grid, runs and compute

Status: Implemented (P1); the full grid runs in P4

Runner script. `scripts/run_synth_grid.py` (engine: `src/experiments/synth_grid.py`) executes resumable units (model, naming, task, strategy, label mode, k , seed). Zero-shot has one unit per (model, naming, task). A query-agnostic unit holds one demonstration set, so all its queries in every environment, plus the content-free query, are scored against one cached prefix; a query-conditional unit builds one prefix per query. The script appends one row per prediction to `results/v3/synthetic/<run>.parquet` once the unit finishes (`RESULTS.COLUMNS_V3` in `src/utils/results_schema.py`):

```
unit_key, run_name, model, model_path, device, device_name, dtype, naming, domain,
task_id, family, env, strategy, mechanism, composition, label_mode, k, seed,
query_id, row_id, demo_ids, demo_labels, order_seed, prompt_hash, n_prompt_tokens,
logprob_0, logprob_1, p1, logprob_0_cf, logprob_1_cf, calibrated_p1, prediction,
prediction_cal, label, y_clean, selection_meta, unit_seconds
```

`demo_ids` lists the pool rows in prompt order and `demo_labels` the labels as shown (they differ from the rows' labels only under shuffled labels). `prediction` is the raw argmax and `prediction_cal` the calibrated one (empty for zero-shot). The file is rewritten through a temporary file and an atomic rename, and a unit is skipped if its rows already exist; a model is not loaded at all when none of its units remain. The P1 smoke run was stopped after 6 of 26 units and rerun: it finished the other 20, with every unit saved exactly once.

Grids. Full grid on Qwen2.5-7B-Instruct. Reduced grid on Qwen2.5-7B base (and on Llama-3.1-8B-Instruct if it is resumed with a fixed answer format): abstract and flipped namings; zero_shot, random, counter_spurious, counter_prior and SATA variants.

P4 runs. `scripts/run_p4.sh` runs resumable stages, in this order:

1. abstract names: zero-shot, random, label_diversity, feature_range, rule_diversity and counter_spurious (3 seeds), the random-label control (label_diversity with shuffled labels), similarity, and the measured pool priors;
2. the same strategies under aligned and flipped names, then their pool priors;
3. counter_prior under every naming;
4. RQ3 on seed 0;
5. the random-label control and similarity under aligned and flipped names;
6. Qwen2.5-7B base: the P2 checks at $k = 8$ on the pilot (label mass, learnability), then, if its median label mass is at least 0.9, its pool priors and the reduced grid (`scripts/p4_base_check.py`);
7. RQ3 on seeds 1 and 2.

Similarity uses one seed: feature kNN selects the same rows for every seed, so extra seeds would only reorder them (decided before P4). The names each task was shown under are recorded with the results (`*.names.json`).

Where P4 ran (1 October 2026). The Mac chain finished the abstract-name grid and stopped partway through the named grid. P4 was then run in full on one Runpod H200 (141 GB, CUDA 13), so every P4 comparison is within one device (`scripts/run_p4_pod.sh`). At batch size 1 one process leaves the GPU mostly idle, so 8 worker processes, each with its own copy of the model and its own shard of units (`--shard i/N`), shared the GPU as clients of NVIDIA MPS: about 160 forward passes per second in total, against about 84 with plain time-slicing and 3.4 on the Mac. The whole of P4

(about 255,000 passes) took 42 minutes. On the H200 the cache check passes (mean AUROC 0.5783 cached against 0.5825 full, difference 0.0042), and Qwen2.5-7B base passes the P2 checks at $k = 8$ (label mass 1.0; $\Delta = 0.103$, $[0.023, 0.186]$). On the 718 units the Mac also ran, prompts are identical, margins correlate at 0.996 and mean AUROC agrees within 0.001, while calibrated predictions agree on only 87% of queries. Results: `results/v3/synthetic/h200/`.

Compute (measured in P2). On the M3 Max with PyTorch on MPS (bf16, batch size 1), a P4 unit (one demonstration set, 60 queries and the content-free query) takes 24, 26 and 32 s at $k = 8, 16, 32$ for Qwen2.5-7B-Instruct and 24, 25 and 31 s for Llama-3.1-8B-Instruct; a zero-shot unit about 23 s; a similarity unit, which builds one prompt per query, about 155 s at $k = 8$ (Qwen). The provisional budget, from timings of the v2 MLX pipeline, assumed about 18.5 s per $k = 8$ unit, so units cost about $1.3\times$ more. The Qwen2.5-7B-Instruct grid at $k = 8$ (3 namings $\times$ 24 tasks) needs 8.7 h for the six query-agnostic strategies at 3 seeds, 0.5 h for zero-shot, and 9.3 h for similarity at 3 seeds or 3.1 h at one seed.

Item	Provisional	Rescaled (P2)
Pilot + prior screening	2.5	7
Qwen2.5-7B-Instruct grid	10	12–18
SATA conditions in the grid	6	8
Reduced grids (Qwen base; Llama paused)	9	6
RQ3	3	4
SATA LLM validation	5	6.5
SATA-MI-mech datamodels	9	12
Probes	3	4
Total	≈ 48	$\approx 60\text{--}66$

The range reflected the number of similarity seeds, fixed at one before P4. Named prompts are about 11% longer (102 against 92 tokens per demonstration line), so named units take slightly longer. With the P3 additions (measured pool priors, the random-label control under every naming and RQ3 on 3 seeds), P4 on the Mac is planned at about 30 hours. Without prefix caching the total would exceed 150 GPU-hours.

40 Gates

Gate	Phase	Criterion	If it fails
P0	P0	<code>pytest tests/</code> passes after porting	fix before P1
G1	P1	pools single-task; no test rows reach selectors; balanced label counts; counter-spurious pooled $ \text{corr}(\sigma_t f_8, y) \leq 0.1$; one BOS on Llama; cached = full scoring on the tiny model (10^{-4}); task label rate in $[0.35, 0.65]$; $\sigma_t f_8$ agreement within ± 0.03 of s (generator scale); prompt snapshot	fix before P2
P2	P2	AUROC(gold labels) – AUROC(shuffled) ≥ 0.05 with interval above 0, at the smallest passing k ; median label mass ≥ 0.9 ; caching moves the mean AUROC by ≤ 0.005	retry with 6 visible features; else abstract names become the floor condition
G3	P3	surrogate $R^2 \geq 0.5$; zero-shot aligned – flipped AUROC ≥ 0.15 (stated in accuracy in the draft)	replace names; else drop naming
5a	P5	oracle targets beat strategies on the proxy; LLM validation run	RQ4 reported as a documented negative
5b	P5	proxy–LLM Spearman ≥ 0.5 ; SATA-MI $\geq$ both controls	report SATA-v2 only
5c	P5	utility $R^2 \geq 0.2$; top- k by utility beats random	skip MI-mech
P6	P6	$ m_L - m \leq 10^{-3}$	fix probe code

Gates passed so far.

- P0, 26 September 2026: `pytest tests/` passed (27 tests).
- G1, 28 September 2026: `pytest tests/` passed (74 tests). The G1 criteria are tested in `tests/test_synth_pipeline.py` (data, selection, grid engine against a stub runner) and `tests/test_hf_runner.py` (prefix caching on the tiny model, a single BOS token, the base-model answer token, prompt snapshots), and the P2 gate’s statistics in `tests/test_gates.py`. The P1 smoke run on Qwen2.5-7B-Instruct and its kill-and-resume check passed.
- P2, 28 September 2026, Qwen2.5-7B-Instruct: learnability passes at $k = 8$ ($\Delta = 0.098$, 95% interval $[0.021, 0.188]$, 9 of 12 tasks) and $k = 16$, and fails at $k = 32$, so $k = 8$; median label mass 0.996; caching moves the mean AUROC by 0.0025. The first P2 run (fixed spurious direction) is archived (Section 12). Llama-3.1-8B-Instruct’s result is not valid (label mass 0.02) and Llama is paused. 80 tests pass.
- G3, 30 September 2026, Qwen2.5-7B-Instruct: **fails as pre-registered**. The surrogate’s out-of-fold R^2 is 0.20 (loan) and 0.28 (medical). The separation passes: zero-shot AUROC aligned minus flipped is 0.164, $[0.053, 0.287]$, on the 12 pilot tasks (0.278 loan, 0.050 medical). The user kept the naming factor, with counter_prior and C_t computed from measured priors, so the surrogate is descriptive only (Section 28; `hiccups/14`). 100 tests pass.
- P4 instrument checks, 1 October 2026, on the Runpod H200: the cache check passes (0.0042), Qwen2.5-7B base passes the P2 checks at $k = 8$, and the H200 reproduces the Mac’s AUROC (within 0.001 on 718 shared units). 104 tests pass.

Part X

Known issues in the v2 implementation

All issues below were verified on branch `synthetic-experiments` before the v3 redesign. No v2 results were saved.

#	Issue	v3 resolution	Phase
1	NB01 merges 5 tasks with different rules into one 1,280-row pool; NB02 draws demonstrations across rules; <code>head(30)</code> evaluates task 0 only	per-task pools; stratified queries	fixed (P1)
2	one fixed seed in every <code>select()</code> call: one demonstration set per strategy for all queries; identical ID data triplicated across datasets	seeded demonstration sets; shared ID data	fixed (P1)
3	counter-spurious proxy search uses <code>test_ood</code> rows and labels (leak); the intended rule reverses f_8 's sign relation; <code>rule_diversity</code> fits a tree on the mixed pool; ground-truth tags not saved	ground-truth tags; magnitude-matched counter-spurious	fixed (P1)
4	prompt renders "...predict Classify the input..."; this specification's v2 quoted a different prompt; its selection and orchestration sections contradicted each other	one template, quoted from real output	fixed (P1)
5	double BOS on Llama (<code>[128000, 128000, ...]</code>)	no added special tokens	fixed (P1)
6	f_8 standard deviation ≈ 1.44 vs 1 for other features	z-scores from pool statistics	fixed (P1)
7	<code>task_meta.json</code> lacks coefficients, thresholds and leaf tables; result rows lack task, seed and demonstration ids	full manifest; results schema	fixed (P1)
8	<code>_regime</code> uses all 3–5 causal features while <code>tree</code> uses 3	exactly 3 load-bearing features	fixed (P1)
9	threshold family has all directions positive; sparse interaction is not monotone	evaluate linear and tree only	fixed (P1)
10	$n = 30$ per cell (interval half-width 0.179)	24 tasks $\times$ 20 queries $\times$ 3 seeds	P4
11	spurious column always at position 9 (positional shortcut for SATA)	column-role permutation	fixed (P1)
12	MiniLM similarity embeds labelled pool rows but unlabelled queries, and its embeddings would change with names	feature-space kNN	fixed (P1)
13	<code>MLXRunner</code> computes logits at every position and has no prefix cache	HFRunner : output layer at the last position only, prefix caching; MLXRunner removed	fixed (P1)
14	NB03 reads a results file that nothing writes and saves to a missing directory	NB03 archived (P1); new analysis notebook	P4
<i>Found while implementing P1</i>			
15	<code>feature_coverage</code> (ported from <code>main</code>) assigned rows feature by feature, so only the top feature ever formed strata	joint quantile cells of the top 3 features	fixed (P1)
16	the ported generator tests sampled tasks from the unseeded global random state, so they failed about one run in six on sampling noise	seeded fixture	fixed (P1)

#	Issue found in P2	Resolution	Phase
17	the bf16 output layer rounded the label logits: margins on a 1/16 grid, 19% of p_1 values tied within a demonstration set	fp32 label logits from the final hidden state (Section 25)	fixed (P2)
18	the spurious direction was the same in every task, so a model prior “bigger values mean 1” backed the shortcut in every task (Qwen’s zero-shot AUROC 0.633, all of it from f_8)	per-task direction σ_t , balanced (Section 12)	fixed (P2)
20	the label-rate filter on the covariate shift accepted almost only shifts of distractors (21 of 24 evaluation tasks), so the environment mostly tested irrelevant features	two load-bearing features shifted in opposite label directions plus one distractor; label rate unchanged (Lemma 11.1)	fixed (P2)
19	Llama-3.1-8B-Instruct began to explain its reasoning (“To infer the rule...”), putting about 2% of its probability on the label tokens, so its scores carried almost no signal	label-mass rule (≥ 0.9) in the gate; Llama paused (a <code>Label:</code> reply prefix fixes the format)	P2

All v2 notebooks that depended on the old design are archived in `notebooks/archive/v2/`; NB01 was rewritten for the per-task design. The data and results of the first P2 run, with the fixed spurious direction, are archived in `data/synthetic/v3/archive/fixed_f8/` and `results/v3/synthetic/archive/fixed_f8/`.

41 Bug-hunting notes (v3 invariants to test)

Items marked (tested) have a test in `tests/`; the rest arrive with the phase that needs them.

1. **Single-task pools.** Every demonstration id belongs to the query’s task (tested, G1).
2. **No test rows upstream.** Selectors, standardisation and priors never see `test_id` or `test_ood` rows (tested for selectors and standardisation; priors in P3).
3. **f_8 is present in every environment.** Only spurious reversal tests reliance on it by flipping its relation to the label.
4. **f_8 statistics on the generator scale.** Sign agreement changes under centring (Lemma 12.3); correlations do not.
5. **Label noise comes after f_8 .** About 2% of rows have f_8 misaligned with the observed label; counter-spurious selection uses the observed label, and both ground-truth strategies exclude label-noise rows from their candidates.
6. **Pairing.** Demonstration seeds exclude the model, naming, label mode and environment, so gold and shuffled units show the same rows (tested with the grid engine).
7. **Mechanism is not evaluated.** It is a placebo for in-context learning (Proposition 11.2); it appears only in SATA training, with matched pools.
8. **Aligned and flipped prompts differ only in three names.** Snapshot-test the system message and the non-load-bearing names.
9. **Balanced composition.** Structured strategies always show $k/2$ labels of each class (tested).
10. **Counter-spurious decorrelation.** Pooled $|\text{corr}(\sigma_t f_8, y)| \leq 0.1$ in counter-spurious sets; naive cell balancing would give about 0.33 (tested).

11. **Covariate shift moves the rule’s inputs.** Two load-bearing features in opposite label directions and one distractor, each by 1–2 standard deviations, with an unchanged label rate (tested). Report which features moved; do not filter shifts on the label rate.
12. **Spurious direction.** σ_t is balanced within each (family, domain) cell, and $\sigma_t = -1$ changes only the sign of f_8 (tested). Statistics of f_8 reliance read f_8 in the task’s direction, and prior-aligned and prior-opposed tasks are reported separately as well as pooled.
13. **Tokenisation.** One BOS on Llama; label tokens single-piece; base-model queries end in "-> " (tested, with the real Qwen and Llama tokenizers).
14. **Prefix caching.** Check the token-prefix condition for every suffix and fall back to a full pass; compare cached and full scoring (tested on the tiny model; P2 on Qwen).
15. **Constrained softmax.** p_0, p_1 renormalise over the two label tokens only; a model confident in some other token can still produce $p_1 \approx 0.5$. This is by design. Mass on other surface forms of the same answer (e.g. " 1") is ignored, a form of surface-form competition (Holtzman et al., 2021).
16. **Output precision.** Compute the two label logits in fp32 from the final hidden state; the bf16 output layer quantises margins to a 1/16 grid (tested: the fp32 path equals the model’s own output in fp32).
17. **Logit lens.** Do not apply the final norm to the last hidden state; it is already normed.
18. **Feature order.** Frozen per task after π_t ; ten columns with single-digit raw names, so alphabetical order equals displayed order. Column roles are read from the manifest (tested).
19. **One device per comparison.** Results record device and dtype; never mix devices within a contrast, because kernels differ slightly across CUDA, MPS and CPU.

References

- Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Workshop Track Proceedings*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=HJ4-rAVt1>.
- Rebecca R. Andridge and Roderick J. A. Little. A review of hot deck imputation for survey non-response. *International Statistical Review*, 78(1):40–64, 2010. URL <https://doi.org/10.1111/j.1751-5823.2010.00103.x>.
- Dana Angluin and Philip Laird. Learning from noisy examples. *Machine Learning*, 2(4):343–370, 1988. URL <https://doi.org/10.1007/BF00116829>.
- Apple Inc. Accelerated PyTorch training on Mac. <https://developer.apple.com/metal/pytorch/>, 2026. Accessed 27 September 2026.
- Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. Invariant risk minimization. arXiv preprint arXiv:1907.02893, 2019. URL <https://arxiv.org/abs/1907.02893>.
- Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, March 2022. URL <https://aclanthology.org/2022.cl-1.7/>.
- Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. arXiv preprint arXiv:2303.08112, 2023. URL <https://arxiv.org/abs/2303.08112>.
- Sebastian Bordt, Harsha Nori, Vanessa Rodrigues, Besmira Nushi, and Rich Caruana. Elephants never forget: Memorization and learning of tabular data in large language models. In *First Conference on Language Modeling (COLM)*, 2024. URL <https://openreview.net/forum?id=HLoWN6m4fS>.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. URL <https://doi.org/10.1023/A:1010933404324>.
- Kay Henning Brodersen, Cheng Soon Ong, Klaas Enno Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *2010 20th International Conference on Pattern Recognition (ICPR)*, pages 3121–3124. IEEE, 2010. URL <https://doi.org/10.1109/ICPR.2010.764>.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- Ting-Yun Chang and Robin Jia. Data curation alone can stabilize in-context learning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8123–8144, Toronto, Canada, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.acl-long.452/>.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’16*, pages 785–794, New York, NY, USA, 2016. ACM. URL <https://doi.org/10.1145/2939672.2939785>.
- Yongqiang Chen, Chenxi Liu, Qingyi Guo, Bo Han, and Kun Zhang. In-context adaptation. In *ICLR 2026 Workshop on Catch, Adapt, and Operate: Monitoring ML Models Under Drift*, Rio de Janeiro, Brazil, 2026. URL <https://openreview.net/forum?id=6deyHb7S8e>.

- Kristy Choi, Chris Cundy, Sanjari Srivastava, and Stefano Ermon. LMPriors: Pre-trained language models as task-specific priors. arXiv preprint arXiv:2210.12530, 2022. URL <https://arxiv.org/abs/2210.12530>. Presented at the NeurIPS 2022 Workshop on Foundation Models for Decision Making.
- William G. Cochran. *Sampling Techniques*. Wiley Series in Probability and Mathematical Statistics. Wiley, New York, 3rd edition, 1977. ISBN 0-471-16240-X. URL <https://www.wiley.com/en-us/Sampling+Techniques,+3rd+Edition-p-9780471162407>.
- Ian Covert, Scott Lundberg, and Su-In Lee. Explaining by removing: A unified framework for model explanation. *Journal of Machine Learning Research*, 22(209):1–90, 2021. URL <https://jmlr.org/papers/v22/20-1316.html>.
- Mark W. Craven and Jude W. Shavlik. Extracting tree-structured representations of trained networks. In David S. Touretzky, Michael C. Mozer, and Michael E. Hasselmo, editors, *Advances in Neural Information Processing Systems*, volume 8, pages 24–30. MIT Press, 1996. URL <https://proceedings.neurips.cc/paper/1995/hash/45f31d16b1058d586fc3be7207b58053-Abstract.html>.
- A. C. Davison and D. V. Hinkley. *Bootstrap Methods and their Application*, volume 1 of *Cambridge Series in Statistical and Probabilistic Mathematics*. Cambridge University Press, Cambridge, 1997. ISBN 0-521-57391-2. URL <https://doi.org/10.1017/CB09780511802843>.
- Luc Devroye, László Györfi, and Gábor Lugosi. *A Probabilistic Theory of Pattern Recognition*, volume 31 of *Applications of Mathematics*. Springer, New York, 1996. ISBN 978-0-387-94618-4. URL <https://doi.org/10.1007/978-1-4612-0711-5>.
- Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. LIFT: Language-interfaced fine-tuning for non-language machine learning tasks. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 11763–11784. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/4ce7fe1d2730f53cb3857032952cd1b8-Abstract-Conference.html.
- W. J. Dixon and A. M. Mood. The statistical sign test. *Journal of the American Statistical Association*, 41(236):557–566, 1946. URL <https://doi.org/10.1080/01621459.1946.10501898>.
- Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. A survey on in-context learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1107–1128, Miami, Florida, USA, November 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.emnlp-main.64/>.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*, volume 57 of *Monographs on Statistics and Applied Probability*. Chapman & Hall, New York, 1993. ISBN 0-412-04231-2. URL <https://doi.org/10.1007/978-1-4899-4541-9>.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. Transformer Circuits Thread, <https://transformer-circuits.pub/2021/framework/index.html>, December 2021. Accessed 27 September 2026.
- Michael Evans and Hadas Moshonov. Checking for prior-data conflict. *Bayesian Analysis*, 1(4): 893–914, 2006. URL <https://doi.org/10.1214/06-BA129>.
- Caoyun Fan, Jidong Tian, Yitian Li, Hao He, and Yaohui Jin. Comparable demonstrations are important in in-context learning: A novel perspective on demonstration selection. In *ICASSP 2024 – 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 10436–10440. IEEE, 2024. URL <https://doi.org/10.1109/ICASSP48485.2024.10448239>.

- Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. URL <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Josh Gardner, Zoran Popović, and Ludwig Schmidt. Benchmarking distribution shift in tabular data with TableShift. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 53385–53432. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a76a757ed479a1e6a5f8134bea492f83-Abstract-Datasets_and_Benchmarks.html. Datasets and Benchmarks Track.
- Shivam Garg, Dimitris Tsipras, Percy S Liang, and Gregory Valiant. What can transformers learn in-context? A case study of simple function classes. In *Advances in Neural Information Processing Systems*, volume 35, pages 30583–30598. Curran Associates, Inc., 2022. URL <https://doi.org/10.52202/068431-2217>.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. URL <https://doi.org/10.1038/s42256-020-00257-z>.
- In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt Cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, pages 325–338, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html.
- Chase Goddard, Lindsay M. Smith, Vudtiwat Ngampruetikorn, and David J. Schwab. When can in-context learning generalize out of task distribution? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 19585–19599. PMLR, 2025. URL <https://proceedings.mlr.press/v267/goddard25a.html>.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Sharut Gupta, Stefanie Jegelka, David Lopez-Paz, and Kartik Ahuja. Context is environment. In *International Conference on Learning Representations*, pages 723–764, 2024. URL <https://openreview.net/forum?id=8VPWfqtQMX>.
- Shivanshu Gupta, Matt Gardner, and Sameer Singh. Coverage-based example selection for in-context learning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 13924–13950, Singapore, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.findings-emnlp.930/>.
- Danny Halawi, Jean-Stanislas Denain, and Jacob Steinhardt. Overthinking the truth: Understanding how language models process false demonstrations. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Tigr1kMDZy>.
- J. A. Hanley and B. J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982. URL <https://doi.org/10.1148/radiology.143.1.7063747>.
- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. URL <https://doi.org/10.1038/s41586-020-2649-2>.
- Hrayr Harutyunyan, Rafayel Darbinyan, Samvel Karapetyan, and Hrant Khachatryan. In-context learning in presence of spurious correlations. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL <https://openreview.net/forum?id=C9CSaTR1iA>.

- Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. TabLLM: Few-shot classification of tabular data with large language models. In Francisco Ruiz, Jennifer Dy, and Jan-Willem van de Meent, editors, *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 5549–5581. PMLR, 2023. URL <https://proceedings.mlr.press/v206/hegselm ann23a.html>.
- Roe Hendel, Mor Geva, and Amir Globerson. In-context learning creates task vectors. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9318–9333, Singapore, December 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.findings-emnlp.624/>.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015. URL <https://arxiv.org/abs/1503.02531>.
- Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. URL <https://doi.org/10.1080/00401706.1970.10488634>.
- Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=cp5PvcI6w8_.
- Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeyer, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. URL <https://doi.org/10.1038/s41586-024-08328-6>.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL <https://www.jstor.org/stable/4615733>.
- Ari Holtzman, Peter West, Vered Shwartz, Yejin Choi, and Luke Zettlemoyer. Surface form competition: Why the highest probability answer isn’t always right. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7038–7051, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. URL <https://aclanthology.org/2021.emnlp-main.564/>.
- Shiyuan Huang, Siddarth Mamidanna, Shreedhar Jangam, Yilun Zhou, and Leilani H. Gilpin. Can large language models explain themselves? A study of LLM-generated self-explanations. arXiv preprint arXiv:2310.11207, 2023. URL <https://arxiv.org/abs/2310.11207>.
- Badr Youbi Idrissi, Martin Arjovsky, Mohammad Pezeshki, and David Lopez-Paz. Simple data balancing achieves competitive worst-group-accuracy. In Bernhard Schölkopf, Caroline Uhler, and Kun Zhang, editors, *Proceedings of the First Conference on Causal Learning and Reasoning*, volume 177 of *Proceedings of Machine Learning Research*, pages 336–351. PMLR, 2022. URL <https://proceedings.mlr.press/v177/idrissi22a.html>.
- Andrew Ilyas, Sung Min Park, Logan Engstrom, Guillaume Leclerc, and Aleksander Madry. Datamodels: Understanding predictions with data and data with predictions. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 9525–9587. PMLR, 2022. URL <https://proceedings.mlr.press/v162/ilyas22a.html>.
- Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, et al. A study of BFLOAT16 for deep learning training. arXiv preprint arXiv:1905.12322, 2019. URL <https://arxiv.org/abs/1905.12322>.

- Mariko Kato, Hakaze Cho, Yoshihiro Sakai, and Naoya Inoue. Affinity and diversity: A unified metric for demonstration selection via internal representations. arXiv preprint arXiv:2502.14380, 2025. URL <https://arxiv.org/abs/2502.14380>.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- Jannik Kossen, Yarin Gal, and Tom Rainforth. In-context learning learns label relationships but is not conventional learning. In *International Conference on Learning Representations*, pages 3209–3280, 2024. URL <https://openreview.net/forum?id=YPIA7bgd5y>.
- Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. Estimating mutual information. *Physical Review E*, 69(6):066138, 2004. URL <https://doi.org/10.1103/PhysRevE.69.066138>.
- Anantha Padmanaban Krishna Kumar. Semantic anchors in in-context learning: Why small LLMs cannot flip their labels. arXiv preprint arXiv:2511.21038, 2025. URL <https://arxiv.org/abs/2511.21038>.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. Measuring faithfulness in chain-of-thought reasoning. arXiv preprint arXiv:2307.13702, 2023. URL <https://arxiv.org/abs/2307.13702>.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set Transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753. PMLR, 2019. URL <https://proceedings.mlr.press/v97/lee19d.html>.
- Itay Levy, Ben Bogin, and Jonathan Berant. Diverse demonstrations improve in-context compositional generalization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1401–1422, Toronto, Canada, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.acl-long.78/>.
- Sichao Li, Xinyue Xu, and Xiaomeng Li. Evaluating LLM understanding via structured tabular decision simulations. arXiv preprint arXiv:2511.10667, 2025. URL <https://arxiv.org/abs/2511.10667>.
- Xiaonan Li, Kai Lv, Hang Yan, Tianyang Lin, Wei Zhu, Yuan Ni, Guotong Xie, Xiaoling Wang, and Xipeng Qiu. Unified demonstration retriever for in-context learning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4644–4668, Toronto, Canada, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.acl-long.256/>.
- Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for GPT-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, pages 100–114, Dublin, Ireland and Online, 2022. Association for Computational Linguistics. URL <https://aclanthology.org/2022.deelio-1.10/>.
- Jiashuo Liu, Tianyu Wang, Peng Cui, and Hongseok Namkoong. On the need for a language describing distribution shifts: Illustrations on tabular datasets. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 51371–51408. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a134eaebd55b7406ff29cd75d5f1a622-Abstract-Datasets_and_Benchmarks.html. Datasets and Benchmarks Track.

- Yang Liu, Sujay Khandagale, Colin White, and Willie Neiswanger. Synthetic benchmarks for scientific research in explainable machine learning. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c16a5320fa475530d9583c34fd356ef5-Abstract-round2.html>.
- Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. Entity-based knowledge conflicts in question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7052–7063, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. URL <https://aclanthology.org/2021.emnlp-main.565/>.
- Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8086–8098, Dublin, Ireland, May 2022. Association for Computational Linguistics. URL <https://aclanthology.org/2022.acl-long.556/>.
- Sewon Min, Xinxu Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11048–11064, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. URL <https://aclanthology.org/2022.emnlp-main.759/>.
- Julian Minder, Kevin Du, Niklas Stoehr, Giovanni Monea, Chris Wendler, Robert West, and Ryan Cotterell. Controllable context sensitivity and the knob behind it. In *International Conference on Learning Representations*, pages 81073–81102, 2025. URL <https://openreview.net/forum?id=Igm9bbkzHC>.
- Jose G. Moreno-Torres, Troy Raeder, Rocío Alaiz-Rodríguez, Nitesh V. Chawla, and Francisco Herrera. A unifying view on dataset shift in classification. *Pattern Recognition*, 45(1):521–530, 2012. URL <https://doi.org/10.1016/j.patcog.2011.06.019>.
- Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do Bayesian inference. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=KSugKcbNf9>.
- Aliakbar Nafar, K. Brent Venable, and Parisa Kordjamshidi. Learning vs retrieval: The role of in-context examples in regression with large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8206–8229, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.naacl-long.417/>.
- Vaishnavh Nagarajan, Anders Andreassen, and Behnam Neyshabur. Understanding the failure modes of out-of-distribution generalization. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=fSTD6NFIW_b.
- Vivian Y. Nastl and Moritz Hardt. Do causal predictors generalize better to new domains? In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. doi: 10.52202/079017-0983. URL <https://arxiv.org/abs/2402.09891>.
- Tai Nguyen and Eric Wong. In-context example selection with influences. arXiv preprint arXiv:2302.11042, 2023. URL <https://arxiv.org/abs/2302.11042>.
- Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. URL <https://doi.org/10.1073/pnas.1708274114>.

- nostalgebraist. interpreting GPT: the logit lens. LessWrong, <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>, August 2020. Accessed 27 September 2026.
- Ryan O’Donnell. *Analysis of Boolean Functions*. Cambridge University Press, Cambridge, 2014. ISBN 978-1-107-03832-5. URL <https://doi.org/10.1017/CB09781139814782>.
- Jane Pan, Tianyu Gao, Howard Chen, and Danqi Chen. What in-context learning “learns” in-context: Disentangling task recognition and task learning. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8298–8319, Toronto, Canada, July 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.findings-acl.527/>.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8026–8037. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.
- Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, second edition, 2009. ISBN 978-0-521-89560-6. URL <https://doi.org/10.1017/CB09780511803161>.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference: Foundations and Learning Algorithms*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA, 2017. ISBN 978-0-262-03731-0. URL <https://mitpress.mit.edu/9780262037310/elements-of-causal-inference/>.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling Transformer inference. In *Proceedings of Machine Learning and Systems*, volume 5, pages 606–624, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.
- Lutz Prechelt. Early stopping – but when? In Genevieve B. Orr and Klaus-Robert Müller, editors, *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer, Berlin, Heidelberg, 1998. URL https://doi.org/10.1007/3-540-49430-8_3.
- Joaquin Quiñonero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D. Lawrence, editors. *Dataset Shift in Machine Learning*. Neural Information Processing Series. MIT Press, Cambridge, MA, 2009. ISBN 978-0-262-17005-5. URL <https://doi.org/10.7551/mitpress/9780262170055.001.0001>.
- Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115, 2024. URL <https://arxiv.org/abs/2412.15115>.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics. URL <https://aclanthology.org/D19-1410/>.

- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “Why should I trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’16, pages 1135–1144. ACM, 2016. URL <https://doi.org/10.1145/2939672.2939778>.
- Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15504–15522, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.acl-long.828/>.
- Elan Rosenfeld, Pradeep Ravikumar, and Andrej Risteski. The risks of invariant risk minimization. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=BbNIbVPJ-42>.
- Brian C. Ross. Mutual information between discrete and continuous data sets. *PLoS ONE*, 9(2): e87357, 2014. URL <https://doi.org/10.1371/journal.pone.0087357>.
- Donald B. Rubin. Matching to remove bias in observational studies. *Biometrics*, 29(1):159–183, 1973. URL <https://doi.org/10.2307/2529684>.
- Ohad Rubin, Jonathan Herzig, and Jonathan Berant. Learning to retrieve prompts for in-context learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2655–2671, Seattle, United States, 2022. Association for Computational Linguistics. URL <https://aclanthology.org/2022.naacl-main.191/>.
- Shiori Sagawa, Pang Wei Koh, Tatsunori B. Hashimoto, and Percy Liang. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=ryxGuJrFvS>.
- Varun Saravanan, Gordon J. Berman, and Samuel J. Sober. Application of the hierarchical bootstrap to multi-level data in neuroscience. *Neurons, Behavior, Data analysis, and Theory*, 3(5):1–25, 2020. URL <https://nbd.scholasticahq.com/article/13927-application-of-the-hierarchical-bootstrap-to-multi-level-data-in-neuroscience>.
- Bernhard Schölkopf, Dominik Janzing, Jonas Peters, Eleni Sgouritsa, Kun Zhang, and Joris Mooij. On causal and anticausal learning. In John Langford and Joelle Pineau, editors, *Proceedings of the 29th International Conference on Machine Learning (ICML)*, pages 1255–1262. Omnipress, 2012. URL <https://icml.cc/2012/papers/625.pdf>.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics. URL <https://aclanthology.org/P16-1162/>.
- Hidetoshi Shimodaira. Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of Statistical Planning and Inference*, 90(2):227–244, 2000. URL [https://doi.org/10.1016/S0378-3758\(00\)00115-4](https://doi.org/10.1016/S0378-3758(00)00115-4).
- Chenglei Si, Dan Friedman, Nitish Joshi, Shi Feng, Danqi Chen, and He He. Measuring inductive biases of in-context learning with underspecified demonstrations. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11289–11310, Toronto, Canada, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.acl-long.632/>.

- Elizabeth A. Stuart. Matching methods for causal inference: A review and a look forward. *Statistical Science*, 25(1):1–21, 2010. URL <https://doi.org/10.1214/09-STS313>.
- Ruixiang Tang, Dehan Kong, Longtao Huang, and Hui Xue. Large language models can be lazy learners: Analyze shortcuts in in-context learning. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 4645–4657, Toronto, Canada, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.findings-acl.284/>.
- Eric Todd, Millicent Li, Arnab Sen Sharma, Aaron Mueller, Byron Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations*, pages 17282–17333, 2024. URL <https://openreview.net/forum?id=AwxytyMwaG>.
- Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. arXiv preprint arXiv:2308.10248, 2023. URL <https://arxiv.org/abs/2308.10248>.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/ed3fea9033a80fea1376299fa7863f4a-Abstract-Conference.html.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- Haochun Wang, Chaofen Yang, Jiatong Liu, Jingbo Wang, Zewen Qiang, Sendong Zhao, Bing Qin, and Ting Liu. Easier to judge than to find: Predicting in-context learning success for demonstration selection. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://openreview.net/forum?id=iixK49TTop>.
- Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained Transformers. In *Advances in Neural Information Processing Systems*, volume 33, pages 5776–5788. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- Jerry Wei, Jason Wei, Yi Tay, Dustin Tran, Albert Webson, Yifeng Lu, Xinyun Chen, Hanxiao Liu, Da Huang, Denny Zhou, and Tengyu Ma. Larger language models do in-context learning differently. arXiv preprint arXiv:2303.03846, 2023. URL <https://arxiv.org/abs/2303.03846>.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, 2020. Association for Computational Linguistics. URL <https://aclanthology.org/2020.emnlp-demos.6/>.
- Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In *International Conference on Learning Representations*, pages 35623–35646, 2024. URL <https://openreview.net/forum?id=auKAUJZM06>.
- Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit Bayesian inference. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=RdJVFCHjUMI>.

- Shangqing Xu and Chao Zhang. Misconfidence-based demonstration selection for LLM in-context learning. arXiv preprint arXiv:2401.06301, 2024. URL <https://arxiv.org/abs/2401.06301>.
- Jiacheng Ye, Zhiyong Wu, Jiangtao Feng, Tao Yu, and Lingpeng Kong. Compositional exemplars for in-context learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 39818–39833. PMLR, 2023. URL <https://proceedings.mlr.press/v202/ye23c.html>.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32, pages 12381–12392. Curran Associates, Inc., 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>.
- Ruiqi Zhang, Spencer Frei, and Peter L. Bartlett. Trained transformers learn linear models in-context. *Journal of Machine Learning Research*, 25(49):1–55, 2024. URL <https://jmlr.org/papers/v25/23-1042.html>.
- Yiming Zhang, Shi Feng, and Chenhao Tan. Active example selection for in-context learning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 9134–9148, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics. URL <https://aclanthology.org/2022.emnlp-main.622/>.
- Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12697–12706. PMLR, 2021. URL <https://proceedings.mlr.press/v139/zhao21c.html>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, pages 62557–62583. Curran Associates, Inc., 2024. URL <https://doi.org/10.52202/079017-2000>.
- Zhiyu Zhu, Zhibo Jin, Jiayu Zhang, Bartłomiej Sobieski, Przemysław Biecek, Fang Chen, and Jianlong Zhou. Faithfulness under the distribution: A new look at attribution evaluation. In *International Conference on Learning Representations (ICLR)*, pages 38289–38312, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/409fcc9d24b549969b8b9be68b56a7be-Abstract-Conference.html.

A Lexicons

Candidates were screened on Qwen2.5-7B-Instruct (Section 28); `configs/lexicons.yaml` holds both lists. Slopes β are from the 800-profile screening run.

Loan approval (1 = approved, 0 = denied).

- Final pairs (raises / lowers approval; contrast $\beta_p - \beta_q$): net worth / collection accounts (2.47); credit score / missed payments (2.29); on-time payment rate / past defaults (1.79); years at current job / recent credit applications (1.45); savings balance / credit card utilisation (1.34).
- Candidates not kept: credit history length / outstanding debt (0.94); collateral value / loan-to-value ratio (0.86); annual income / debt-to-income ratio (0.20).
- Final weak names: form version; mailing batch number; office floor number; loan officer ID; days since form printed; applicant ID last digit; hour of submission ($|\beta| \leq 0.32$).
- Weak candidates not kept: branch number; application day of month; pages in application (β 0.36 to 0.71).

Medical risk (1 = high risk, 0 = low risk).

- Final pairs (raises / lowers risk): systolic blood pressure / weekly exercise hours (1.62); waist circumference / grip strength (0.67); resting heart rate / fitness test score (0.65); LDL / HDL cholesterol (0.65); smoking pack-years / nightly hours of sleep (0.29).
- Candidates not kept: body mass index / daily vegetable servings (0.21); fasting blood glucose / daily step count (-0.07); alcohol drinks per week / lung function score (-0.28).
- Final weak names: printer tray number; staff ID digit; clinic room number; form page count; chart version; waiting room seat number; minutes since check-in (β 0.06 to 0.52).
- Weak candidates not kept: appointment weekday; referral code digit; record batch number (β 0.72 to 0.76).

B The eight admissible tree tables

Exhaustive enumeration over all 2^8 leaf tables finds 16 that are complement antisymmetric, of which exactly 8 pass the degeneracy filter, and each equals a signed majority. Each table lists the labels of leaves 0 to 7, with leaf index $L = b_1 + 2b_2 + 4b_3$.

Leaf table	Signs (s_1, s_2, s_3) of $\text{Maj}(\ell_1, \ell_2, \ell_3)$
00010111	$(+, +, +)$
00101011	$(-, +, +)$
01001101	$(+, -, +)$
01110001	$(+, +, -)$
10001110	$(-, -, +)$
10110010	$(-, +, -)$
11010100	$(+, -, -)$
11101000	$(-, -, -)$